



Electric truck-based robot delivery problem with nonlinear charging

Shaohua Yu ^{a,*}, Tengkuo Zhu ^b, Jakob Puchinger ^{c,d}

^a School of Intelligence Science and Technology, Nanjing University of Science and Technology, Nanjing, 210094, China

^b The University of Texas at Austin, Austin, Texas, US

^c EM Normandie Business School, Métis Lab, 92110, Clichy, France

^d Université Paris-Saclay, CentraleSupélec, Laboratoire Génie Industriel, 91190, Gif-Sur-Yvette, France

ARTICLE INFO

Keywords:

Logistics
Nonlinear charging
Truck-based robot delivery
Electric vehicle routing
Adaptive large neighborhood search

ABSTRACT

This study addresses an electric truck-based robot collaborative delivery problem that incorporates the inherently nonlinear charging behavior of both vehicle types. A mixed-integer linear programming model with piecewise linear charging representation is developed, and its solution approach is enhanced by time-dependent valid inequalities and tailored heuristics. Numerical experiments show that the proposed methods efficiently generate high-quality solutions across diverse urban logistics scenarios. The results indicate that partial charging strategies consistently outperform full charging, and that assuming linear charging may yield infeasible or misleading solutions in practice. Sensitivity analyses further reveal that system performance is most influenced by battery capacity and the number of charging stations, whereas variations in charging rates have comparatively minor effects.

1. Introduction

Promoting low-carbon and green logistics has become a major trend in urban distribution. Compared to traditional fuel trucks, electric trucks stand out with their zero emissions in urban areas and fewer restrictions, gradually becoming an important transportation tool in logistics distribution (Alarcón et al., 2023). In addition, accelerating the development of smart logistics, actively applying modern information technology and intelligent equipment, and improving the level of logistics automation, unmanned operation, and intelligence is one of the main directions of the logistics distribution industry today (Turan et al., 2024). In recent years, significant advancements in artificial intelligence, sensor technology, and the Internet of Things have accelerated the innovation and practical deployment of robotic systems in logistics distribution (Chung, 2021). Robot-driven logistics distribution has become a key aspect of intelligent upgrading within logistics enterprises, playing a transformative role in reshaping their delivery operations (Alverhed et al., 2024; Bakach et al., 2021; Srinivas et al., 2022).

However, electric trucks and robots still face challenges when delivering alone. Electric trucks face restrictions when operating in certain environments, such as narrow streets or campuses. Meanwhile, robots encounter limitations related to speed, operational range, and carrying capacity, making it challenging for them to independently handle medium- and long-distance logistics tasks (Boysen et al., 2018). In addition, if electric trucks and robots need to work for long periods of time, they still need to be recharged. This work proposes a new model of electric truck-based robot collaborative distribution: the electric truck serves as a mobile platform for transporting and supplying robots, and tasks are executed both sequentially and in parallel by the electric truck and robot,

* Corresponding author.

E-mail addresses: shaohua.yu@njust.edu.cn (S. Yu), zhutengkuo@utexas.edu (T. Zhu), jpuchinger@em-normandie.fr (J. Puchinger).

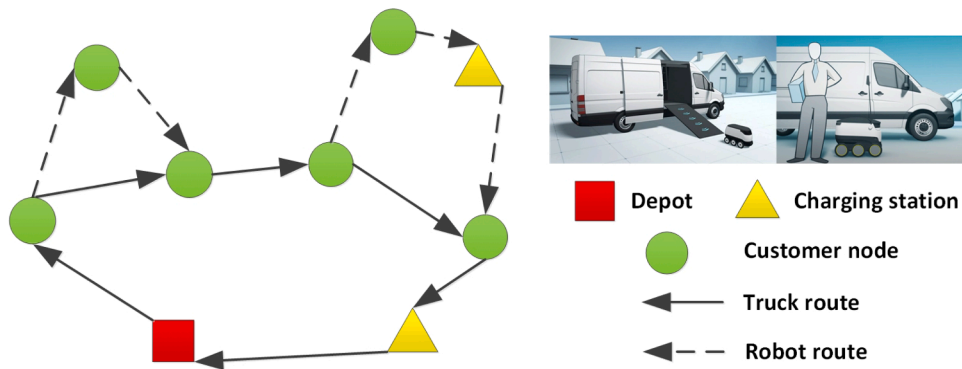


Fig. 1. A simple example of the route of electric truck-based robot collaborative distribution.

effectively avoiding their respective defects, increasing the flexibility of the distribution system, and thereby improving distribution efficiency. A simple example of the route of electric truck-based robot collaborative distribution is shown in Fig. 1: the electric truck carries the robot or the electric truck travels alone on the first-level route, serving customers during the period, and the robot serves customers along the second-level open route. The electric truck releases or recycles the robot, and replenishes the goods carried by the robot when needed at the nodes they meet. The electric truck and robot can be charged at the charging station. Compared with drones, robots do not rely on low-altitude airspace operations and can be more readily integrated into existing road-side and last-mile delivery environments. Therefore, truck-robot collaboration represents not merely a variant of truck-drone delivery, but a practically relevant delivery mode with distinct deployment potential.

With the continuous development of energy storage and charging technology, vehicle charging modes are becoming more diversified. For example, under the full charging mode, electric vehicles need to be fully charged each time, while the partial charging mode is the opposite. In addition, due to the influence of battery characteristics and other factors, the vehicle charging process shows nonlinear characteristics, manifested as nonlinear functions (Liu et al., 2025). Multiple charging modes and nonlinear charging functions make the system's power management complex, bringing new challenges to the application of electric truck-based robot collaborative distribution (Wang et al., 2025).

Therefore, this paper will investigate the electric truck-based robot collaborative delivery problem under nonlinear charging functions, aiming to explore how to model and efficiently solve the collaborative delivery issue in the context of realistic charging functions. Our study will consider the following two charging modes under nonlinear charging functions: the first is the full charging mode, where vehicles must fully charge their batteries upon arrival at charging stations. In the decision-making process, we need to take into account the nonlinear relationship between battery capacity and charging time. Notably, once the decision to charge a vehicle is made, there is no need to determine the amount or duration of charging. The second mode is partial charging, where vehicles can charge their batteries to any desired level upon reaching charging stations. Again, the nonlinear relationship between battery capacity and charging time must be considered in decision-making. Additionally, we need to determine the amount of electricity or the charging duration for the vehicle. We will draw on the comparison framework used in Schneider et al. (2014), Hiermann et al. (2016) to assess the impact of the two charging modes on the output of the electric truck-based robot collaborative delivery model under nonlinear charging conditions. It is worth noting that our study primarily focuses on the partial charging mode. This is because, both theoretically and practically, the full charging mode is essentially a special case of the partial charging mode. To isolate the effect of nonlinear charging and avoid conflating it with fleet-size expansion, this study focuses on a setting with a single truck carrying a single robot. This allows the core charging mechanism to be examined more clearly, while still providing a methodological foundation for future multi-truck, multi-robot extensions.

The main contributions of this paper are as follows: First, we introduce an electric truck-robot collaborative delivery problem that considers a nonlinear charging function (ETRD-NL). Our model assumes that the truck and the robot have separate batteries, which better reflects real-world scenarios and fundamentally differs from the study by Zhu et al. (2025). Second, we use a piecewise linear approximation to transform the nonlinear charging function into a mixed-integer linear programming (MILP) model. To enhance solution efficiency, we design a set of valid inequalities tailored to the model's characteristics. Experimental results show that these improvements significantly accelerate the solution process, achieving a substantial speedup. Third, considering the computational challenges of obtaining an exact solution, we propose two “sequence first, charge second” heuristics and a “simultaneous sequencing and charging” algorithm. The latter provides high-quality solutions efficiently. Finally, through extensive simulation experiments, we systematically compare partial charging and full charging strategies, using linear and nonlinear charging functions, in terms of delivery efficiency. The findings offer valuable theoretical insights and practical guidance for decision-making in different charging strategies.

The remainder of this paper is structured as follows: In Section 2, we review the relevant literature; Section 3 describes the problem background, model assumptions, and mathematical formulation; Section 4 presents our solution methods and algorithm design; Section 5 shows the numerical experiment results along with detailed analysis; and finally, Section 6 concludes the paper.

2. Literature review

In this section, we will first introduce the truck-based robot delivery problem, followed by a discussion on the electric vehicle routing problem and the related charging functions.

2.1. Truck-based robot delivery

Truck-based robot delivery systems conceptualize the truck as a mobile base for deploying and retrieving delivery robots (Yu and Puchinger, 2024). Given the operational similarities between truck-robot and truck-drone (Luo et al., 2025) collaborative logistics, both are herein collectively referred to as truck-based robot delivery systems. These systems can be classified based on the number of trucks and the number of robots each truck accommodates: (i) a single truck with one robot (Murray and Chu, 2015; Carlsson and Song, 2018); (ii) a single truck with multiple robots (Kang and Lee, 2021; Cavani et al., 2021); and (iii) multiple trucks coordinating, either each with a single robot (Yu et al., 2022, 2025) or with multiple robots (Ostermeier et al., 2023; Yu et al., 2020; Ostermeier and Huf, 2025).

Recent advancements in truck-based robot delivery have incorporated various operational constraints and service settings. Time window constraints have been extensively explored, including hard (Chen et al., 2021) and soft variants (Ostermeier et al., 2023). In addition, service heterogeneity has been addressed through customer segmentation models, where certain customer groups (e.g., the elderly) prefer human-assisted delivery over autonomous systems (Heimfarth et al., 2022).

Further research has extended the model to accommodate operational flexibility, such as en-route charging of electric robots via the truck (Yu et al., 2024), dual-service (pickup and delivery) configurations (Yu et al., 2022), and centralized facilities such as depots (Ostermeier et al., 2022) and hubs (Yu et al., 2023) for robot reallocation and energy optimization.

The truck-based robot delivery problem is primarily addressed using mathematical programming and constraint programming techniques. Mixed-integer linear programming remains the predominant approach, enabling formal modeling of routing and synchronization, often augmented by valid inequalities to enhance computational efficiency (Tamke and Buscher, 2021). More recently, constraint programming has emerged as a viable alternative due to its flexible constraint specification and dynamic handling of complex scenarios (Ham, 2018). Nonetheless, both approaches face scalability challenges as problem size and constraint dimensionality increase, often leading to prohibitive computation times in large-scale or real-time applications.

It is important to highlight that truck-based robots share similarities with truck-based (flying) drones (Wang et al., 2024; Madani et al., 2024; Han et al., 2025). Although the two systems differ in certain operational aspects, such as movement characteristics and distance calculation, these distinctions are not the main methodological focus of this study. Instead, this paper concentrates on a more fundamental issue arising in truck-robot collaboration, namely, how independent battery management under nonlinear charging affects coordinated routing and scheduling decisions. In particular, when ground robots are used as collaborative delivery units, their charging behavior and energy constraints become tightly coupled with truck routing, service assignment, and timing synchronization. Very recently, novel applications combining trucks, drones and robots have been studied in the literature (Moghadam et al., 2025; Yang et al., 2025).

This paper examines the scenario of an electric truck carrying a robot, taking into account partial charging models and the implications of nonlinear charging functions.

2.2. Electric vehicle routing problem

Electric vehicle routing problems arise from the limited range and charging requirements of battery-powered vehicles, necessitating strategic integration of charging stops into route planning (Qin et al., 2021). These problems can be broadly categorized into several types: electric vehicle traveling salesman problem (Doppstadt et al., 2020), mixed electric vehicle routing problem (Seyfi et al., 2022; Wang and Zhao, 2023), electric vehicle location-routing problem (Hulagu and Celikoglu, 2021; Aghalari et al., 2023), multi-type electric vehicle routing problem (Wang et al., 2025), electric taxi problem (Tang et al., 2020), two-tier electric vehicle routing problem (Jie et al., 2019), and electric vehicle pickup and delivery problem (Goeke, 2019; Zhou et al., 2024).

The challenge in evaluating decision-making for electric vehicle routing problems lies in developing charging strategies under various charging modes, charging functions, and problem characteristics (Xie et al., 2023). The introduction of multiple charging modes presents new challenges for the design of charging strategies for electric vehicles. In a full charging mode, electric vehicles must charge completely each time, whereas in a partial charging mode, the charging strategy can be rule-based (Desaulniers et al., 2016). In reverse charging mode, vehicles can charge each other, allowing energy transfer, with current research employing heuristic methods or solvers for making charging decisions (Park et al., 2025). The nonlinear charging function creates a nonlinear relationship between vehicle charging and multiple factors, complicating energy management for electric vehicles. Existing studies often use piecewise linear approximations and other methods to handle nonlinear functions, thereby designing charging strategies (Zhou et al., 2022).

For example, Montoya et al. (2017) designed a MILP model to address partial charging strategies, nonlinear charging functions, various charging technologies, and complex electric vehicle routing problems with multiple charging stations. Froger et al. (2019) introduced two enhanced MILP models. The first model uses arc-indexed variables instead of vertex-indexed variables to track travel time and distance. The second model relies on a charging path paradigm, where paths represent the sequence of stops at charging stations between nodes. Froger et al. (2022) further considered the issue of charging station capacity based on Montoya et al. (2017) and established a MILP model, solving it using commercial solvers. Yamín et al. (2025) addressed the electric vehicle routing and

overnight charging scheduling problems on multiple graphs, also considering nonlinear charging functions, and solved them using branch-and-price methods. In a recent article, Wang et al. (2025) address the heterogeneous-fleet electric vehicle routing problem with nonlinear charging functions. The authors formulate the problem as a path-based mixed-integer linear program and introduce a method that combines iterated local search with a set-partitioning model. Extensive computational experiments show that accounting for nonlinear charging functions can notably lower the total cost.

A new fast approach that simultaneously optimizes both EV routes and charging schedules, delivering exact solutions within seconds while accounting for the nonlinear charging behavior of EVs, has been developed by Liu et al. (2025). As in previous research, the nonlinear charging process is approximated with a piecewise linear function. The charging schedule is modeled as a Mixed-Integer Programming problem to maximize time efficiency along a fixed route. Station selection is linked to charging amount decisions and integrated with route planning through a State-Space-Time network. This allows for simultaneous optimization of routes and charging schedules. Computational experiments based on real-world data demonstrate the efficiency of the proposed approach.

2.3. Research gap

Despite significant progress in the integration of electric trucks and autonomous delivery robots, notable research gaps remain.

First, charging strategies—particularly for heterogeneous battery systems—are often oversimplified. The model proposed by Yu et al. (2024) assumes a linear charging behavior, overlooking real-world (nonlinear) effects.

Second, while nonlinear charging functions have been incorporated in electric vehicle routing (Montoya et al., 2017; Froger et al., 2019), such considerations are rarely extended to truck-robot collaboration models. For example, Zhu et al. (2025) assumed a shared battery between drones and trucks, which is an operationally impractical assumption. In reality, trucks and robots are equipped with independent energy systems and may require asynchronous, location-specific charging.

This study addresses these gaps by proposing a novel optimization model for electric truck-based robot delivery with nonlinear charging functions. Efficient solution algorithms are also developed to ensure computational tractability in medium/large-scale applications.

3. Problem description and model

3.1. Problem statement

In this study, we focus on ETRD-NL problem. This problem involves an electric truck and its onboard robot, both of which can independently visit customer nodes. The customers can also be visited by the truck while carrying the robot. The objective is to minimize the maximum completion time. Additionally, the problem includes charging stations that can be visited by either the truck or the robot individually (autonomous robot energy replenishment is technically feasible Saputra et al., 2024), or by the truck while carrying the robot. Both the truck and the robot have separate batteries, and the charging process at these stations follows a nonlinear function, meaning that simultaneous charging does not result in a simple additive increase in charging efficiency. Furthermore, we assume partial charging, allowing the battery to be charged to any level rather than requiring it to be fully charged. Note that autonomous robot charging only means that the robot can recharge at stations; it does not eliminate the efficiency advantage of collaboration between the truck and the robot.

The problem formulation does not incorporate time window constraints. We assume that the robot is capable of servicing multiple customers sequentially on a single trip before returning to the truck, and customer service time is not explicitly modeled but rather integrated into the travel time of an arc, provided it is a fixed value. Moreover, all customer nodes used for dropping off and picking up the robot must be visited by the truck, and both the truck and the robot are allowed to meet at charging stations. Also, the same charging station can be accessed only once by the same type of vehicle (truck/robot).

A key assumption in our model is the synchronization constraint: if both the truck and the robot arrive at a certain node, they must be coordinated in time, meaning that they must be present at that node simultaneously. This assumption is made to simplify the model.

3.2. Variable and parameter definitions

To formally define the ETRD-NL model, we introduce the necessary notation. The problem is defined on a network represented by a set of nodes $\mathcal{V}$, which includes the start depot 0, the end depot $0'$, the set of customer locations $\mathcal{V}_c$, and the set of charging stations $\mathcal{V}_s$. We also define a combined set $\mathcal{V}_c^s = \mathcal{V}_c \cup \mathcal{V}_s$. The potential movements between nodes are captured by the arc set $\mathcal{A} = \{(0, j) \mid j \in \mathcal{V}_c^s\} \cup \{(i, j) \mid i, j \in \mathcal{V}_c^s, i \neq j\} \cup \{(i, 0') \mid i \in \mathcal{V}_c^s\}$. Associated with each arc $(i, j) \in \mathcal{A}$ are parameters representing the travel time (t_{ij}^T for the truck, t_{ij}^R for the robot) and energy consumption (c_{ij}^T for the truck, c_{ij}^R for the robot). Note that if the robot is carried by the truck on arc (i, j) , its energy consumption c_{ij}^R is zero. The truck and the robot possess battery capacities Q^T and Q^R , respectively. The demand q_i at each customer node $i \in \mathcal{V}_c$ is treated as a counter, set to 1. We utilize a constant $C = 1$ and a large constant M in the formulation.

The core decision variables define the vehicle routes: x_{ij} , y_{ij} , and z_{ij} are binary variables indicating if arc (i, j) is traversed by the truck, the robot, or the truck carrying the robot, respectively. Variables w_i^T and w_i^R track the arrival times at node i for the truck and robot. Energy levels are monitored via e_i^T and e_i^R (remaining battery upon arrival at node i) and E_i^T and E_i^R (remaining battery upon departure from node i). The duration spent charging at a station $i \in \mathcal{V}_s \cup \{0, 0'\}$ is represented by o_i^T for the truck and o_i^R for the

Table 1
Variable and parameter definitions.

Symbol	Description
<i>Sets and Indices</i>	
$\mathcal{V}_0$	Set containing the start depot 0 and end depot 0'.
$\mathcal{V}_s$	Set of charging stations.
$\mathcal{V}_c$	Set of customers.
$\mathcal{V}_c^s$	Set of customers and charging stations, $\mathcal{V}_c \cup \mathcal{V}_s$.
$\mathcal{V}$	Set of all nodes, $\mathcal{V}_0 \cup \mathcal{V}_c \cup \mathcal{V}_s$.
$\mathcal{A}$	Set of feasible arcs for truck/robot routes, $\mathcal{A} = \{(0, j) \mid j \in \mathcal{V}_c^s\} \cup \{(i, j) \mid i, j \in \mathcal{V}_c^s, i \neq j\} \cup \{(i, 0') \mid i \in \mathcal{V}_c^s\}$.
$\mathcal{L}$	Set of indices for piecewise linear segments of the charging function, $\mathcal{L} = \{1, 2, \dots, L\}$.
<i>Parameters</i>	
– General Parameters –	
t_{ij}^T, t_{ij}^R	Travel time between nodes i and j for the truck and robot, respectively.
c_{ij}^T, c_{ij}^R	Energy consumption of the truck and robot on arc (i, j) , respectively; $c_{ij}^R = 0$ if robot is carried by truck.
Q^T, Q^R	Battery capacity of the truck and robot, respectively.
q_i	Demand at customer node $i \in \mathcal{V}_c$. Used as a counter, $q_i = 1$.
C, M	Fixed constant ($C = 1$) and a sufficiently large positive number (Big-M).
– Charging Function Approximation Parameters ($l \in \mathcal{L}$) –	
k_l^T, k_l^R	Slope of the l th linear segment for the truck's/robot's charging function.
b_l^T, b_l^R	Intercept of the l th linear segment for the truck's/robot's charging function.
c_l^T, c_l^R	State of charge (SoC) breakpoint defining the end of the l th segment for truck/robot.
t_l^T, t_l^R	Charging time required to reach SoC c_l^T (truck) or c_l^R (robot).
<i>Decision Variables</i>	
– Routing and Scheduling –	
x_{ij}	1 if arc (i, j) is traveled by the truck, 0 otherwise.
y_{ij}	1 if arc (i, j) is traveled by the robot, 0 otherwise.
z_{ij}	1 if arc (i, j) is traveled by the truck with the robot onboard, 0 otherwise.
w_i^T, w_i^R	Arrival time of the truck/robot at node $i \in \mathcal{V}$.
u_i	Auxiliary variable for subtour elimination at node $i \in \mathcal{V}_c^s$.
– Energy and Charging –	
e_i^T, e_i^R	Remaining battery level of the truck/robot upon arrival at node $i \in \mathcal{V}$.
E_i^T, E_i^R	Remaining battery level of the truck/robot upon departure from node $i \in \mathcal{V}$.
z_i^T, z_i^R	Projected time to full charge from current battery level for truck/robot at node $i \in \mathcal{V}_s \cup \mathcal{V}_0$.
o_i^T, o_i^R	Actual charging duration for the truck/robot at node $i \in \mathcal{V}_s \cup \mathcal{V}_0$.
– Piecewise Linear Approximation Auxiliary Variables ($l \in \mathcal{L}$) –	
p_{il}^T	1 if truck's arrival energy e_i^T at station i falls in segment l range $[c_{l-1}^T, c_l^T]$, 0 otherwise. Assumes $c_0^T = 0$.
p_{il}^R	1 if robot's arrival energy e_i^R at station i falls in segment l range $[c_{l-1}^R, c_l^R]$, 0 otherwise. Assumes $c_0^R = 0$.
P_{il}^T	1 if truck's departure energy E_i^T from station i falls in segment l range $[c_{l-1}^T, c_l^T]$, 0 otherwise. $c_0^T = 0$.
P_{il}^R	1 if robot's departure energy E_i^R from station i falls in segment l range $[c_{l-1}^R, c_l^R]$, 0 otherwise. $c_0^R = 0$.

robot. The variables z_i^T and z_i^R are related to charging time calculation: time from current battery to full charge for truck and robot at node $i \in \mathcal{V}_s \cup \mathcal{V}_0$. The variable u_i is used for sequencing and subtour elimination.

To model the nonlinear charging process, we employ a piecewise linear approximation defined over a set of segments $\mathcal{L} = \{1, 2, \dots, L\}$. For each segment $l \in \mathcal{L}$, the charging function for the truck is characterized by its slope k_l^T , intercept b_l^T , charge level breakpoint c_l^T , and corresponding time breakpoint t_l^T . Analogous parameters ($k_l^R, b_l^R, c_l^R, t_l^R$) define the robot's charging function. Binary variables p_{il}^T (p_{il}^R) and P_{il}^T (P_{il}^R) are introduced to identify the active charging segment based on the battery level upon arrival (e_i^T/e_i^R) and departure (E_i^T/E_i^R) at charging station $i \in \mathcal{V}_s$.

The detailed information about the notations is provided in Table 1.

3.3. Linearization of nonlinear charging function

The ETRD-NL considers heterogeneous vehicles (truck and robot) with distinct battery characteristics, necessitating separate nonlinear charging functions. To handle the nonlinear charging function in the electric truck-based robot collaborative delivery problem, we employ piecewise linear (PWL) approximation. This approximation enables the model to be efficiently solved by mainstream commercial solvers without disrupting the MILP structure; meanwhile, by customizing the segmentation to control the approximation error, and ensuring conservativeness (not underestimating charging time, thus avoiding infeasible paths), monotonicity, and convexity, it enhances the stability and interpretability of the solutions. In terms of generality, PWL is a mature approach for handling nonlinear charging in electric transportation optimization and can be seamlessly extended to various VRP variants (e.g., Montoya et al., 2017; Froger et al., 2019; Kullman et al., 2021b,a; Froger et al., 2022; Yamín et al., 2025; Zhu et al., 2025).

The formulation details are presented for the truck (using superscript T); the corresponding model for the robot is analogous, simply substituting the superscript T with R . Fig. 2 illustrate typical nonlinear charging curves approximated by linear segments for vehicles (the curves of the truck and robot refer to this figure).

The piecewise linear function is defined by a set of breakpoints (t_l^T, c_l^T) for $l \in \mathcal{L} = \{1, 2, \dots, L\}$, where t_l^T is the time required to charge from empty to the state of charge c_l^T . We assume $c_0^T = 0$ and $t_0^T = 0$, and typically $c_L^T = Q^T$ represents the full battery capacity

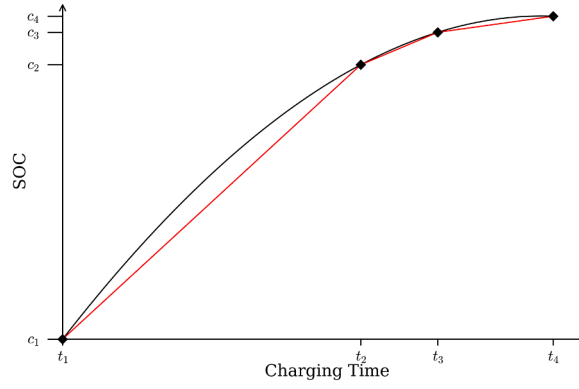


Fig. 2. Illustrative nonlinear charging function approximation for vehicle.

reached at time t_L^T . Each linear segment l , spanning the charge interval $[c_{l-1}^T, c_l^T]$, has a constant charging rate represented by its slope k_l^T and an intercept b_l^T . Note that the indices used in constraints might reference the interval $[c_l^T, c_{l+1}^T]$ depending on the exact formulation.

Eq. (1) formally relates the breakpoints for each segment $l \in \mathcal{L}$:

$$c_l^T = k_l^T \times t_l^T + b_l^T, \quad \forall l \in \mathcal{L} \quad (1)$$

We also use the same method to represent robots nonlinear charging function, which will not be elaborated here.

3.4. Mixed linear integer programming model

Then we construct the mathematical model for partial charging mode as follows, and the full-charging model can be obtained as a special case of the partial-charging model.

Objective function

$$\min w_{0'}^T \quad (2)$$

The objective function (2) is defined as minimizing the total time to serve all customers.

Arc Constraints

$$\sum_{(i,j) \in \mathcal{A}} x_{ij} \leq 1, \quad \forall j \in \mathcal{V}_c^s \quad (3)$$

$$\sum_{(i,j) \in \mathcal{A}} x_{ij} - \sum_{(j,i) \in \mathcal{A}} x_{ji} = 0, \quad \forall j \in \mathcal{V}_c^s \quad (4)$$

$$\sum_{(i,j) \in \mathcal{A}} y_{ij} \leq 1, \quad \forall j \in \mathcal{V}_c^s \quad (5)$$

$$\sum_{(i,j) \in \mathcal{A}} y_{ij} - \sum_{(j,i) \in \mathcal{A}} y_{ji} = 0, \quad \forall j \in \mathcal{V}_c^s \quad (6)$$

$$\sum_{i \in \mathcal{V}_c^s} x_{i0'} = \sum_{j \in \mathcal{V}_c^s} x_{0j} = \sum_{i \in \mathcal{V}_c^s} y_{i0'} = \sum_{j \in \mathcal{V}_c^s} y_{0j} = \sum_{i \in \mathcal{V}_c^s} z_{i0'} = \sum_{j \in \mathcal{V}_c^s} z_{0j} = 1 \quad (7)$$

$$\sum_{(i,j) \in \mathcal{A}} y_{ij} + \sum_{(i,j) \in \mathcal{A}} x_{ij} \geq 1, \quad \forall j \in \mathcal{V}_c \quad (8)$$

$$2 \times z_{ij} \leq x_{ij} + y_{ij} \leq z_{ij} + 1, \quad \forall (i,j) \in \mathcal{A} \quad (9)$$

$$x_{ij}, y_{ij}, z_{ij} \in \{0, 1\}, \quad \forall (i,j) \in \mathcal{A} \quad (10)$$

Constraints (3-4) guarantee that each node is visited by the truck no more than once, and that the truck departs from every node it has visited. Constraints (5-6) apply the same rules to the robot. Constraints (7) ensure that the number of departures and returns to the depot by the truck is equal to the number of departures and returns by its onboard robot, and equal to 1. And note when leaving from the depot and returning to the depot, the robot must be carried by the truck. Constraints (8) stipulate that each customer node must be visited at least once, either by the truck or by the robot. Constraints (9) linearize the logical relationship $z_{ij} = x_{ij} \wedge y_{ij}$. They ensure that z_{ij} equals 1 if and only if both x_{ij} and y_{ij} are 1. This signifies that the arc (i,j) is traversed by the truck with the robot onboard. Finally, constraints (10) are variable constraints.

Time Constraints

$$\max(w_i^T + o_i^T, w_i^R) + t_{ij}^T - w_j^T \leq M(1 - x_{ij}), \quad \forall (i, j) \in \mathcal{A} | i \in \mathcal{V}_s \cup \{0\} \quad (11)$$

$$\max(w_i^R + o_i^R, w_i^T) + t_{ij}^R - w_j^R \leq M(1 - y_{ij} + x_{ij}), \quad \forall (i, j) \in \mathcal{A} | i \in \mathcal{V}_s \quad (12)$$

$$\max(w_i^T, w_i^R) + t_{ij}^T - w_j^T \leq M(1 - x_{ij}), \quad \forall (i, j) \in \mathcal{A} | i \in \mathcal{V}_c \cup \{0\} \quad (13)$$

$$\max(w_i^R, w_i^T) + t_{ij}^R - w_j^R \leq M(1 - y_{ij} + x_{ij}), \quad \forall (i, j) \in \mathcal{A} | i \in \mathcal{V}_c \quad (14)$$

$$|w_j^T - w_j^R| \leq M \left(1 - \sum_{(i,j) \in \mathcal{A}} z_{ij} \right), \quad \forall j \in \mathcal{V}_c^s \cup \{0'\} \quad (15)$$

$$|w_i^T - w_i^R| \leq M(1 - z_{ij}), \quad \forall (i, j) \in \mathcal{A} | i \in \mathcal{V}_c^s \cup \{0\} \quad (16)$$

Constraints (11)–(16) are time flow constraints. Constraints (11) represent the time flow constraints on the truck route starting from either a charging station or the depot. Constraints (12) represent the time flow constraints on the robot route starting specifically from a charging station (excluding the depot). Constraint (13) represents the time flow constraints on the truck route starting from either a customer node or the depot. Constraint (14) represents the time flow constraints on the robot route starting specifically from a customer node. Constraint (15) ensures that in the case of a truck carrying a robot, the truck and the robot arrive at a certain node at the same time. Constraint (16) ensures that if the truck and robot depart from a node i together to travel along arc (i, j) (i.e., $z_{ij} = 1$), their times at node i (w_i^T and w_i^R) must be synchronized.

Energy Constraints

$$e_j^T + c_{ij}^T - E_i^T \leq M(1 - x_{ij}), \quad \forall (i, j) \in \mathcal{A} | i \in \mathcal{V}_s \cup \{0\} \quad (17)$$

$$e_j^T + c_{ij}^T - e_i^T \leq M(1 - x_{ij}), \quad \forall (i, j) \in \mathcal{A} | i \in \mathcal{V}_c \cup \{0\} \quad (18)$$

$$e_j^R + c_{ij}^R - E_i^R \leq M(1 - y_{ij} + x_{ij}), \quad \forall (i, j) \in \mathcal{A} | i \in \mathcal{V}_s \quad (19)$$

$$e_j^R + c_{ij}^R - e_i^R \leq M(1 - y_{ij} + x_{ij}), \quad \forall (i, j) \in \mathcal{A} | i \in \mathcal{V}_c \quad (20)$$

$$e_j^R - E_i^R \leq M(1 - z_{ij}), \quad \forall (i, j) \in \mathcal{A} | i \in \mathcal{V}_c^s \cup \{0\} \quad (21)$$

$$0 \leq e_i^T, E_i^T \leq Q^T, \quad \forall i \in \mathcal{V} \quad (22)$$

$$0 \leq e_i^R, E_i^R \leq Q^R, \quad \forall i \in \mathcal{V} \quad (23)$$

$$c_l^T - M \times (1 - p_{il}^T) \leq e_i^T \leq c_{l+1}^T + M \times (1 - p_{il}^T), \quad \forall i \in \mathcal{V}_s, l \in \mathcal{L} \quad (24)$$

$$c_l^R - M \times (1 - p_{il}^R) \leq e_i^R \leq c_{l+1}^R + M \times (1 - p_{il}^R), \quad \forall i \in \mathcal{V}_s, l \in \mathcal{L} \quad (25)$$

$$c_l^T - M \times (1 - P_{il}^T) \leq E_i^T \leq c_{l+1}^T + M \times (1 - P_{il}^T), \quad \forall i \in \mathcal{V}_s, l \in \mathcal{L} \quad (26)$$

$$c_l^R - M \times (1 - P_{il}^R) \leq E_i^R \leq c_{l+1}^R + M \times (1 - P_{il}^R), \quad \forall i \in \mathcal{V}_s, l \in \mathcal{L} \quad (27)$$

$$\sum_{l \in \mathcal{L}} p_{il}^T = \sum_{l \in \mathcal{L}} p_{il}^R = \sum_{l \in \mathcal{L}} P_{il}^R = \sum_{l \in \mathcal{L}} P_{il}^T = 1, \quad \forall i \in \mathcal{V}_s \quad (28)$$

$$k_l^T \times (t_L^T - z_i^T) + b_l^T - M(1 - p_{il}^T) \leq e_i^T \leq k_l^T \times (t_L^T - z_i^T) + b_l^T + M(1 - p_{il}^T), \quad \forall i \in \mathcal{V}_s, l \in \mathcal{L} \quad (29)$$

$$k_l^R \times (t_L^R - z_i^R) + b_l^R - M(1 - p_{il}^R) \leq e_i^R \leq k_l^R \times (t_L^R - z_i^R) + b_l^R + M(1 - p_{il}^R), \quad i \in \mathcal{V}_s, \forall l \in \mathcal{L} \quad (30)$$

$$k_l^T \times (t_L^T - z_i^T + o_i^T) + b_l^T - M(1 - P_{il}^T) \leq E_i^T \leq k_l^T \times (t_L^T - z_i^T + o_i^T) + b_l^T + M(1 - P_{il}^T), \quad i \in \mathcal{V}_s, \forall l \in \mathcal{L} \quad (31)$$

$$k_l^R \times (t_L^R - z_i^R + o_i^R) + b_l^R - M(1 - P_{il}^R) \leq E_i^R \leq k_l^R \times (t_L^R - z_i^R + o_i^R) + b_l^R + M(1 - P_{il}^R), \quad i \in \mathcal{V}_s, \forall l \in \mathcal{L} \quad (32)$$

Constraints (17)–(32) are energy flow constraints. Constraints (17) are energy flow constraints on the truck route from a charging station i or depot. Constraints (18) are energy flow constraints on the truck route from a customer node i or depot. Similarly, Constraints (19–20) are energy flow constraints on the robot route from a charging station or a customer node i , respectively. Constraints (21) represent that if the robot is carried by the truck along arc (i, j) (i.e., $z_{ij} = 1$), the robot's energy at node j cannot exceed its energy when departing node i . Constraints (22–23) are battery variable constraints. Constraints (24–25) determine the interval in which the remaining power e_i^T and e_i^R of the current electric truck and robot is located, as well as the corresponding function expression. Constraints (26–27) determine the corresponding segment for the energy levels after charging (E_i^T and E_i^R). Constraints (28) represent variable constraints indicating that for each charging station $i \in \mathcal{V}_s$, exactly one segment is selected for each of the four variable sets $p_{il}^T, p_{il}^R, P_{il}^T, P_{il}^R$.

Constraints (29)–(32) apply the selected linear function to establish the relationship between charging time and battery level. Constraints (29) and (30) link the arrival energy (e_i) to the arrival time (z_i). Constraints (31) and (32) link the post-charging energy (E_i) to the departure time (arrival time z_i plus charging duration o_i). The linear function, defined by slope k_l and intercept b_l , approximates the state of charge based on the time spent in the charging process.

Miller-Tucker-Zemlin (MTZ) Constraints

To eliminate sub-tours within the routing problem, we introduce the MTZ constraints. These constraints ensure a sequential visit order, thereby preventing the formation of cycles that do not include the depot. For each node $i \in \mathcal{V}_c^s$ (where $\mathcal{V}_c^s$ denotes the set of customer and charging station nodes), we define an order variable u_i indicating the position of node i in the routing sequence. The variables are bounded as follows:

$$1 \leq u_i \leq |\mathcal{V}_c^s| \quad \forall i \in \mathcal{V}_c^s \quad (33)$$

To ensure that the truck's and robot's routes are free from sub-tours, the following MTZ constraints are added to the model:

$$u_i - u_j + |\mathcal{V}_c^s| \times x_{i,j} \leq |\mathcal{V}_c^s| - 1 \quad \forall i, j \in \mathcal{V}_c^s, i \neq j \quad (34)$$

$$u_i - u_j + |\mathcal{V}_c^s| \times y_{i,j} \leq |\mathcal{V}_c^s| - 1 \quad \forall i, j \in \mathcal{V}_c^s, i \neq j \quad (35)$$

3.5. Valid inequalities

$$\sum_{(i,j) \in A} t_{ij}^T \times x_{ij} \leq w_{0'}^T, \quad (36)$$

$$\sum_{(i,j) \in A} t_{ij}^T \times z_{ij} + \sum_{(i,j) \in A} t_{ij}^R \times (y_{ij} - z_{ij}) \leq w_{0'}^R, \quad (37)$$

Constraints (36)–(37) provide minimum time requirements. Constraints (36) specify that the arrival time of each truck at the depot must not be less than the total time needed to complete its assigned route, including the time required to serve all the customers it is responsible for. Likewise, constraints (37) define the minimum time required for a robot onboard the truck.

4. Methodology

In this section, we present two complementary solution paradigms for the ETRD-NL problem.

(1) Sequence First, Charging Second: a two-phase, decomposed approach. This approach consists of two steps: (i) a (meta)heuristic is used to generate a feasible truck-based robot route while ignoring energy constraints; and (ii) given the resulting fixed route, we insert charging-station visits into the customer sequence and determine the optimal charging duration at each stop. To accomplish the second step, we adapt both exact (frvcpy Kullman et al., 2021a) and heuristic techniques for the fixed-sequence vehicle charging problem. We emphasize that the frvcpy method cannot be directly applied to a truck-based robot delivery route, so customized settings are required.

(2) Simultaneous Sequencing and Charging: a unified approach that makes routing and charging decisions in tandem. By coupling these decisions, this method explores a richer neighborhood of solutions.

4.1. Decision: sequencing first

In the STC framework, the solution procedure is decomposed into two modules: a sequencing module that generates a TSPR route and a charging module that completes the route by inserting charging decisions. To isolate the effect of the sequencing mechanism, we consider two sequence generators under the same STC charging module: an adapted Savings-based sequencing procedure as the benchmark and the proposed ALNS-based sequencing procedure.

The sequencing decision part is divided into two stages: (i) initial route generation and (ii) route improvement through the improvement methods. The output of this stage is a “good” Traveling Salesman Problem with Robot route (TSPR route), which serves as the basis for the subsequent charging decision.

4.1.1. ALNS-based sequencing procedure

We begin by obtaining an initial route via the classical Traveling Salesman Problem (TSP) solution using the simple genetic algorithm. Subsequently, we transform the TSP route into a TSPR route tailored for cooperative truck-based robot delivery. To this end, Algorithm 1 is applied. In brief, the algorithm proceeds in three stages. First, the initial TSP route is divided into several overlapping segments with the specified split probability using the function SPLITWITHOVERLAP. The purpose of overlapping is to

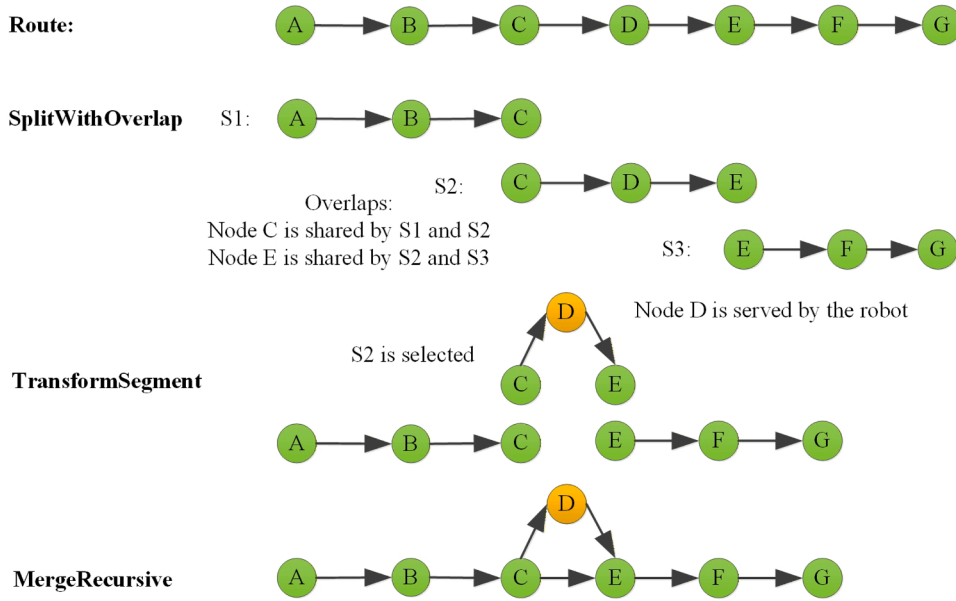


Fig. 3. Example of the operators in the GENERATE TRUCK-ROBOT ROUTE algorithm.

preserve connectivity information between adjacent segments. Note **SPLITWITHOVERLAP** is used to split TSP route (ordered list of nodes) into sublists with overlapping endpoints. For instance, the sequence [a, b, c, d, e, f, g] would be split into [a, b, c], [c, d, e], and [e, f, g]. Second, each segment is converted to a preliminary TSPR segment via **TRANSFORMSEGMENT**, with the split probability controlling transformation variability (boundary segments always fully transformed). Note **TRANSFORMSEGMENT** is used to convert each TSP segment into a TSPR segment. Third, adjacent segments are recursively merged using a function to form the overall TSPR route, which is called **MERGE RECURSIVE**. Fig. 3 is a simple example illustrating this three-stage conversion process from an initial TSP route to a final cooperative TSPR route.

Algorithm 1 GenerateTruck-RobotRoute.

Require: L : TSP route (ordered list of nodes) p : split probability, $0 \leq p \leq 1$

Ensure: R : cooperative truck-robot route (ordered list of nodes)

```

1:  $S \leftarrow \text{SPLITWITHOVERLAP}(L)$  ▷ Split  $L$  into sublists with overlapping endpoints
2:  $T \leftarrow []$ 
3: for  $i \leftarrow 1$  to  $|S|$  do
4:   if  $i = 1$  or  $i = |S| - 1$  then
5:      $\rho \leftarrow 1$ 
6:   else
7:      $\rho \leftarrow p$ 
8:   end if
9:    $T \leftarrow T \cup \text{TRANSFORMSEGMENT}(S_i, \rho)$  ▷ Convert each TSP segment into a TSPR segment
10: end for
11:  $R \leftarrow \text{MERGE RECURSIVE}(T)$  ▷ Recursively merge adjacent segments and remove duplicates
12: return  $R$ 

```

The initial route is further improved through an ALNS framework. This framework relies on the combination of various destroy and repair operators. The overall iteration strategy is as follows: At each iteration, a destroy operator is selected to remove a subset of nodes. Subsequently, based on the chosen destroy operator, a repair operator is selected to re-insert these nodes, potentially leading to a route with an improved objective value.

We distinguish between two types of customer nodes:

- **Joint Customer Nodes:** refer to customer nodes where the truck releases or receives robots.
- **Regular Customer Nodes:** refer to all customer nodes except for the Joint Customer Nodes.

The specific operators are designed as follows:

- **Random Regular Customer Node Removal:** Randomly remove a fraction (α ranging from 15% to 50%) of regular customer nodes.

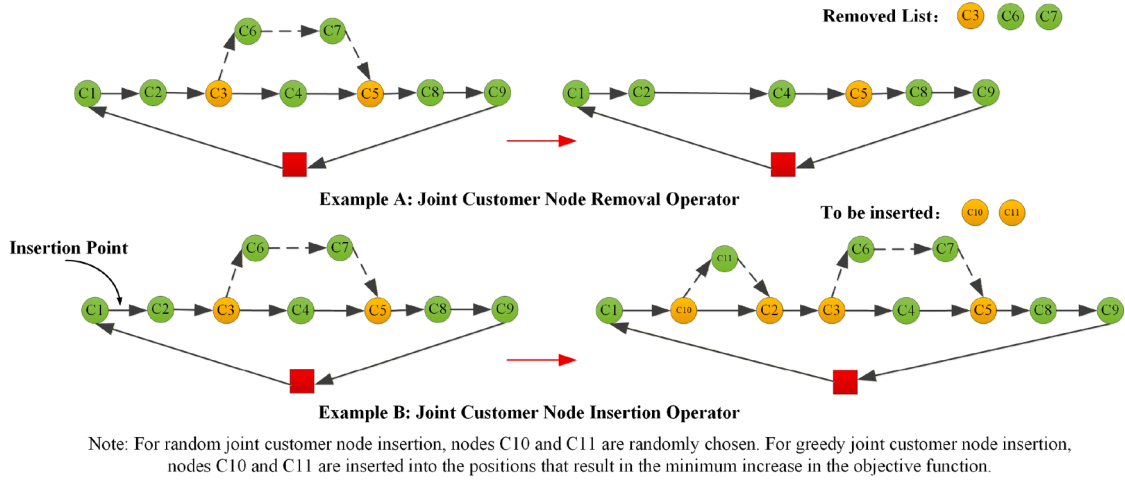


Fig. 4. Example of joint customer node removal/insertion operator.

- **Greedy Regular Customer Node Removal:** Sequentially removes regular customer nodes that have the highest cost contribution. The cost contribution of a node is the reduction in the objective function value achieved by its removal.
- **Random Joint Customer Node Removal:** Randomly remove a fraction (α ranging from 15% to 50%) of joint customer nodes (simultaneously eliminating the corresponding robot routes). When a joint node is removed, the synchronization structure at that location is dissolved. The truck route segments adjacent to the joint node are merged (i.e., the truck bypasses the removed node), and the associated robot route is dismantled, with all its visited customers added to the removal pool for re-insertion.
- **Greedy Joint Customer Node Removal:** Systematically remove joint customer nodes (simultaneously eliminating the corresponding robot routes) that provide the largest cost savings to the overall objective function. When a joint node is removed, the synchronization structure at that location is dissolved. The truck route segments adjacent to the joint node are merged (i.e., the truck bypasses the removed node), and the associated robot route is dismantled, with all its visited customers added to the removal pool for re-insertion.
- **Random Regular Customer Node Insertion:** Reinsert removed regular customer nodes randomly into any feasible positions.
- **Greedy Regular Customer Node Insertion:** Reinsert removed regular customer nodes sequentially at positions that minimally increase the objective function.
- **Random Joint Customer Node Insertion:** Randomly reinserts joint customer nodes by constructing a small parallel substructure and embedding it into the current solution. Specifically, we randomly select an eligible insertion location in an edge (i, j) on a regular truck carries robot route. The selected connection is removed and replaced by a parallel substructure consisting of a joint customer node and one robot customer node.
- **Greedy Joint Customer Node Insertion:** Sequentially reinsert joint customer nodes at the positions determined by the minimum increase in the objective function.
- **Regret Insertion Regular Node Operator:** This operator applies the regret-k heuristic to insert a regular customer node into feasible positions in the route. For each candidate position, it calculates the increase in cost (regret value) if the node is not inserted at its best position, and selects the position with the highest regret to maximize solution quality.
- **Regret Insertion Joint Node Operator:** This operator uses the regret-k heuristic to insert a joint charging station node into the route, considering both truck and robot schedules within parallel segments. For each candidate position, the regret value is evaluated, and the position with the highest regret is chosen for insertion.

To facilitate a clearer understanding of these operations, we provide a simple example of the Joint Customer Node Removal/Insertion Operator in Fig. 4.

An adaptive mechanism based on the *Destroy-Based Repair Weight-Adjusted* paradigm (Yu et al., 2023) is used for operator selection. Moreover, a non-improvement iteration strategy (Pisinger and Ropke, 2018) is deployed to prevent the search process from becoming trapped in local optima. The detailed ALNS algorithm framework can be seen in Appendix A.

4.1.2. Savings-based sequencing baseline (Savings-STC)

To benchmark the proposed ALNS-based sequencing procedure, we also implement a simpler Savings-based sequencing baseline within the same STC framework, denoted as Savings-STC. This baseline is adapted from the classical FSTSP savings idea of Murray and Chu (2015), but is modified to fit our truck-based robot collaborative routing setting.

The procedure starts from a pure truck-served TSP route, in which all customers are initially visited by the truck. Then, the algorithm iteratively examines whether removing a customer from the truck route and reassigning it can reduce the current objective

value. Different from the original FSTSP, where a drone typically serves only one customer per sortie, our setting allows a robot to visit multiple customers consecutively during one dispatch. Therefore, when a customer is removed from the truck route, three types of reassignment are considered: (1) inserting the customer into an existing robot sub-route, (2) reinserting the customer into another position on the truck route, or (3) using the customer as the first node of a newly created robot sub-route.

For each candidate reassignment, we compute the resulting change in the objective value and interpret it as the corresponding saving. The best feasible move is accepted only if it yields a positive improvement. This process is repeated until no further improving reassignment can be found. In this way, the initial truck-only route is gradually converted into a cooperative TSPR route. Since the charging-completion phase is exactly the same as that used in ALNS-STC, the comparison between the two methods isolates the effect of the sequencing strategy alone.

4.2. Decision: charging second

Once we have the sequence of customer visits (from Section 4.1), we need to figure out when, where, and how much the truck and robot should charge their batteries. This means adding charging stations to the route and deciding when and how much to charge at each station to make sure neither the truck nor the robots run out of power.

The power levels of the truck and robot do not affect each other, and during the sequencing stage, their respective routes are already determined. From an operational perspective, independently addressing the charging needs of the truck and robot seems to be a simple and feasible approach. Therefore, we propose a simple heuristic method: independent energy optimization for the truck and robot routes.

4.2.1. Charging only what's needed

Electric vehicle batteries, like those in our truck and robot described in Fig. 2, usually charge faster at the beginning and charging slows down as the battery gets fuller. Because of this nonlinear charging behavior, the best way to save time is only to charge exactly enough energy at a station to reach the next place charging is needed (either the next charging station on the route or the final depot) just as the battery runs out. Charging more than necessary at one station means carrying extra energy and spending longer charging time (to get the extra energy) than needed. We formally state this efficient charging principle as follows:

Proposition 4.1 (Charge-Enough Principle). A vehicle must visit a fixed, predetermined sequence of charging stations.¹ Let the cumulative charging-time function be $T : [0, Q] \rightarrow \mathbb{R}_{\geq 0}$ with $T(0) = 0$, $T'(s) > 0$, $T''(s) \geq 0$; i.e. T is strictly increasing and convex, so the marginal charging time grows with the SoC. Under these conditions, the strategy that minimises the total time spent charging is:

at every station where charging occurs, add exactly the amount of energy needed to reach the next charging station (or the depot) with zero energy remaining.

The proof is provided in Appendix B. Note that in the subsequent heuristics, we will use this approach in optimizing charging.

4.2.2. Sequential charging plan heuristic

In the truck-based robot delivery problem, the robot depends on the truck for transportation, release, and retrieval. Therefore, prioritizing the optimization of truck energy (such as setting up charging stations along the truck's route) is a wise choice. Hence, we propose the following charging methods:

Method 1: exact truck route charging then heuristic robot route charging. First, we look only at the truck's route derived from the TSPR sequence. We use methods designed for planning charging stations on a fixed route, the exact method: frvcpy library (Kullman et al., 2021a), to add charging station visits where the truck needs them, resulting in a truck route that minimizes charging time. Next, based on the established truck route, we employ a heuristic method (Algorithm 2), with Proposition 4.1, to greedily insert charging stations into the robot's route, which consists of multiple segments. Ultimately, we will obtain a collaborative delivery route that includes charging stations for the truck and the robot.

The Algorithm 2 first inputs a complete TSPR route P , along with a dictionary C that includes corresponding charging times for the existing charging stations and the initial data for both the truck and the robot. Next, it loops through each segment of the TSPR route and performs the following actions (lines 2–18). For each segment, the algorithm first calculates the truck's end time (T_{truck}) and end energy (E_{truck}) (line 3). Based on the obtained E_{truck} (if the segment is a truck carrying a robot, consider it; else, ignore it), the algorithm can also determine the charging time of the truck at the charging station, which is the same for the robot. The algorithm then calculate the robot's end time (T_{robot}) and energy (E_{robot}) (line 4).

If T_{truck} is greater than T_{robot} , algorithm find out how much additional time it can give the robot route without increasing the objective value of the TSPR route (this is called the 'gap' (Δ)). If the robot's energy does not meet its needs, the algorithm repairs the route while considering the 'gap time' (lines 8). Otherwise, the algorithm tries to insert a charging station and charge until the 'gap time' is used up (lines 10). If T_{robot} is greater than T_{truck} , the algorithm repairs the robot route's energy if needed (lines 13). Finally, the algorithm updates variables and parameters (lines 15–16) before proceeding to the next segment.

¹ The sequence may terminate at the depot, which can itself be regarded as the last charging station.

Algorithm 2 GreedyInsertRobotCS.

Require: P : original TSPR route (ordered list of nodes). C : charging-station $\rightarrow$ charging-time map. T_{truck}^0, E_{truck}^0 : start time and energy of truck. T_{robot}^0, E_{robot}^0 : start time and energy of robot.

Ensure: P' : repaired TSPR route with charging stations inserted.

```

1:  $P' \leftarrow P$ 
2: for each segment  $s$  in  $P'$  do
3:    $(T_{truck}, E_{truck}) \leftarrow \text{CALCULATETRUCKENDSTATE}(s, T_{truck}^0, E_{truck}^0, C)$   $\triangleright$  Compute end-of-segment time & energy for truck
4:    $(T_{robot}, E_{robot}) \leftarrow \text{CALCULATEROBOTENDSTATE}(s, T_{robot}^0, E_{robot}^0, E_{truck})$   $\triangleright$  Compute robot's end time & energy, note if segment is
      truck carrying robot, consider truck's end energy; else, ignore it
5:   if  $T_{truck} > T_{robot}$  then
6:      $\Delta \leftarrow T_{truck} - T_{robot}$   $\triangleright$  Time gap to be filled by charging
7:     if  $E_{robot} < 0$  then
8:        $P' \leftarrow \text{REPAIRROUTEENERGY}(P', s, \Delta, C)$   $\triangleright$  Insert minimum stations to satisfy feasibility
9:     else
10:       $P' \leftarrow \text{INSERTCHARGINGUNTILGAP}(P', s, \Delta, C)$   $\triangleright$  Optionally use idle time for extra charging
11:    end if
12:  else
13:     $P' \leftarrow \text{REPAIRROUTEENERGYIFNEEDED}(P', s, C)$   $\triangleright$  Truck arrives earlier or on-time
14:  end if
15:   $(T_{truck}^0, E_{truck}^0) \leftarrow (T_{truck}, E_{truck})$ 
16:   $(T_{robot}^0, E_{robot}^0) \leftarrow (T_{robot}, E_{robot})$ 
17: end for
18: return  $P'$ 

```

Method 2: heuristic truck route charging then heuristic robot route charging. We start by applying a heuristic method (see Algorithm 3) to insert charging stations along the truck's route, ensuring it has enough energy to complete its journey. Once the truck route is feasible, we then use Algorithm 2 to insert charging stations into the robot's route segments.

Algorithm 3 is the algorithm to insert truck charging stations greedily.

Algorithm 3 GreedyInsertTruckCS.

Require: Fixed truck route $R = (v_1, v_2, \dots, v_{|R|})$; initial state-of-charge e_0

Ensure: Augmented truck route $\bar{R}$ with charging stations **or** "Infeasible"

```

1:  $e \leftarrow e_0$   $\triangleright$  current energy
2: for  $k \leftarrow 1$  to  $|R| - 1$  do  $\triangleright$  scan every segment
3:    $\delta \leftarrow \Delta(v_k, v_{k+1})$   $\triangleright$  energy needed for  $v_k \rightarrow v_{k+1}$ 
4:   if  $e < \delta$  then  $\triangleright$  not enough energy?
5:     inserted  $\leftarrow$  false
6:     for  $h \leftarrow k - 1$  downto 1 do  $\triangleright$  search backward
7:       if STATIONFEASIBLEAFTER( $v_h$ ) then
8:         INSERTSTATIONAFTER( $v_h$ )
9:          $e \leftarrow e + E_{chg}$ 
10:        inserted  $\leftarrow$  true
11:        break
12:      end if
13:    end for
14:    if not inserted then
15:      return INFEASIBLE
16:    end if
17:  end if
18:   $e \leftarrow e - \delta$   $\triangleright$  consume energy
19: end for
20: return  $\bar{R}$ 

```

4.2.3. Exploring further optimization opportunities

We use exact methods or heuristic approaches to independently optimize the energy consumption of the truck route or the robot route. However, the very interaction between the truck and robot, especially during parallel segments, introduces coordination complexities that might lead these methods to overlook potentially superior solutions. The focus on inserting charging stations primarily based on immediate energy needs might miss strategic uses of the charging station network.

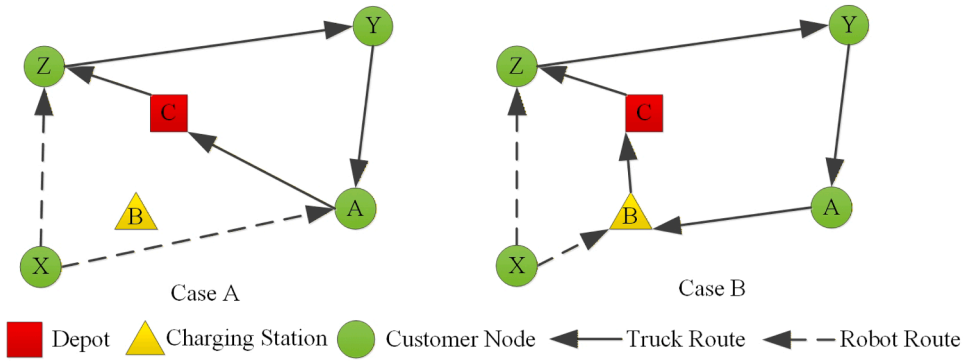


Fig. 5. Example of further optimization.

Consider a scenario illustrated in Fig. 5: even if neither vehicle urgently needs recharging, stopping briefly at a charging station could offer an ideal rendezvous node. A faster vehicle could pause briefly (perhaps even performing a small beneficial recharge, as suggested by Proposition 4.1) to synchronize effectively with the slower vehicle. This strategic waiting could reduce idle time later in the journey, ultimately improving the total completion time (makespan). Our existing methods might not explicitly identify or exploit such strategic opportunities unless a vehicle explicitly requires charging.

Therefore, while our current heuristic methods are effective initial solutions, they might not comprehensively explore the entire solution space. To uncover these more sophisticated interactions and potentially achieve further reductions in total time or cost, we now introduce advanced optimization techniques designed to examine the solution space more thoroughly.

Specifically, we've developed two additional optimization procedures to further enhance the solutions generated by route sequencing and charging decisions. First, we try to insert regular charging nodes greedily: Insert regular charging stations at positions that cause the smallest increase in the objective function. Second, we try to insert a joint charging node greedily: (1) Insert joint charging stations (stations handling both vehicles simultaneously) along with their associated parallel-route segments into the gaps between existing regular and parallel routes. (2) Insert joint charging stations and their corresponding parallel-route segments directly within regular routes.

Note that after insertion, we remove unnecessary charging stations whose elimination most significantly improves the objective function.

Each insertion is done greedily, selecting the position that minimally increases the objective function. After insertion, we again remove unnecessary charging stations whose removal provides the greatest improvement in the objective function. Note: The first insertion strategy explicitly incorporates the greedy rule we previously proposed.

4.3. Decision: simultaneous sequencing and charging

As mentioned in the previous section, the “sequence first, charging second” approach has inherent shortcomings in the truck-based robot delivery problem that need to be addressed. In this section, we make decisions regarding both sequencing and charging simultaneously, thereby avoiding the limitations imposed by a restricted solution space. Specifically, we will keep the previously introduced four destroy and six repair operators, and introduce the following six operators into the ALNS operator pool:

- **Random Regular Charging Node Removal:** Randomly selects and removes one regular charging station visit from the current solution.
- **Random Joint Charging Node Removal:** Randomly selects and removes one joint charging station, and the nodes on the robot route associated with this charging station.
- **Random Regular Charging Node Insertion:** Attempts to insert one regular charging station into feasible positions within the route randomly. Feasibility checks include energy constraints. Then, using Random Regular Customer Node Insertion operator to insert customers. If the route is infeasible, call the Repair Energy operator to repair energy.
- **Random Joint Charging Node Insertion:** Attempts to insert one joint charging station stop into feasible positions, potentially requiring adjustments to both truck and robot schedules within parallel segments. Feasibility checks include energy constraints. Then, using Random Regular Customer Node Insertion operator to insert customers. If the route is infeasible, call the Repair Energy operator to repair energy.
- **Greedy Regular Charging Node Insertion:** Greedy to insert one regular charging station into the route with feasible positions to minimize the objective increase. Feasibility checks include energy constraints. Then, using Random Regular Customer Node Insertion operator to insert customers. If the route is infeasible, call the Repair Energy operator to repair energy.
- **Greedy Joint Charging Node Insertion:** Greedy to insert one joint charging station stop into feasible positions, potentially requiring adjustments to both truck and robot schedules within parallel segments. Feasibility checks include energy constraints. Then, using Random Regular Customer Node Insertion operator to insert customers. If the route is infeasible, call the Repair Energy operator to repair energy.

By incorporating these operators, the ALNS framework is able to intelligently add, remove, or relocate charging stations as part of its neighborhood exploration, in addition to modifying customer sequences. Note that after a repair operator, a feasibility check is performed. If an energy violation is detected, a repair energy procedure is triggered. If the repair fails, the solution is discarded. The Feasibility Check Operator and Repair Energy Operator are as follows:

- **Feasibility Check Operator:** Simulates the truck and robot moving along the route step by step to verify feasibility under energy constraints. At each visited node, it updates the state of charge of both vehicles and assumes that whenever a vehicle visits a charging station, it is fully recharged. If, even under this full-recharge assumption, some part of the route cannot be completed, the solution is marked as infeasible and passed to the Repair Energy operator.
- **Repair Energy Operator:** Attempts to restore feasibility when a route violates the energy constraints by combining the ideas of Algorithms 2 and 3. For single-vehicle segments (e.g., the truck route), it follows the Algorithm 3 backtracking rule: it scans the route, and whenever the remaining energy is not sufficient to reach the next node, it moves backward along the route and inserts one or more charging stations until the segment becomes feasible or no valid insertion is possible. For parallel segments where the truck and robot move independently, it applies the Algorithm 2 synchronization rule: at the end of each segment, it compares the arrival times of the truck and robot, treats any earlier robot arrival as “free” waiting time, and attempts to insert robot charging stops within this gap so that recharging does not increase the overall makespan. If no sufficient waiting gap exists or feasibility still cannot be restored, it reverts to Algorithm 3-style backtracking on the robot route.

Given the complexity of the problem, it is essential to further refine solutions through targeted local search procedures. Therefore, after completing a destroy and a repair operation, the algorithm integrates two types of local search operators to improve solution quality: Node Local Search and Structure Local Search.

- **Node Local Search Operator:** This operator greedily improves the overall route structure by swapping node positions within each route segment. Specifically, for each segment and each route, all pairs of nodes are considered for intra-route swaps. A feasibility check is conducted after each swap, and the new solution is accepted only if it is feasible and yields a better objective value. The process iterates until no further improvements can be achieved through node exchanges.
- **Structure Local Search Operator:** This operator targets the unique joint node that connects two adjacent route segments and applies a symmetric interchange strategy to perform local optimization. The procedure involves attempting to swap the joint node with candidate nodes selected from both the current segment and the preceding segment. Each swap is followed by a feasibility check, and the new solution is accepted if it is feasible and improves the objective value. This operator allows flexible adjustment of the connection between vehicles and robots at the route junctions, thereby further optimizing the route structure.

5. Computational study

This section presents a comprehensive computational study to rigorously evaluate the proposed ALNS algorithm, validate the MILP formulation, and analyze the impact of different charging strategies and system parameters on the ETRD-NL problem. We first examine the performance of the ALNS algorithm by comparing the effectiveness of several charging decision strategies and sequence strategies. We then validate and benchmark the MILP formulation by investigating how the inclusion of valid inequalities affects the solution time for tiny and small-scale instances solved with CPLEX, and by comparing the results of the ALNS-based heuristic with the exact MILP solutions on these instances. Finally, we analyze the impact of alternative charging policies (e.g., partial, full, and linear charging), and conduct a sensitivity analysis on key system parameters, including truck and robot battery capacities, charging rates, and the number of available charging stations.

The mathematical programming model uses the Docplex Python library and the CPLEX 12.8 solver with Python 3.6.8. The ALNS algorithm is implemented in Python 3.6.8 as well. Both the Docplex model and the ALNS algorithm were executed on a machine with two 2.55 GHz AMD EPYC 7Y43 processors (64 cores each; one thread pinned for Python and 32 for CPLEX) and 256 GB of RAM under Windows Server 2019. Unless otherwise stated, the time limit for solving the MILP model with CPLEX is 7200 s. Each instance of the ALNS algorithm was executed for 200,000 iterations. To guarantee the statistical reliability of the findings, each experiment was replicated 10 times with distinct random seeds.

5.1. Instances generation

We adopt the geographic coordinates and experiment parameters presented in Yu et al. (2024) as the foundation for our computational studies. The Manhattan distance metric is employed to measure the distances between every pair of nodes in the network. To thoroughly examine our approach, we construct four distinct categories of problem instances, each containing 20 scenarios: tiny-scale (7 customers, 3 charging stations), small-scale (15 customers, 7 charging stations), medium-scale (30 customers, 15 charging stations), and large-scale (60 customers, 30 charging stations). Specifically, the tiny-scale instances are generated within a spatial area of $10 \times 10 \text{ km}^2$, subdivided into blocks of $100 \times 100 \text{ m}^2$. The small-scale instances, in contrast, utilize a $10 \times 10 \text{ km}^2$ region partitioned similarly into $100 \times 100 \text{ m}^2$ blocks. For medium-scale scenarios, we expand the spatial scope to a $20 \times 20 \text{ km}^2$ area, also composed of blocks sized at $100 \times 100 \text{ m}^2$. The large-scale instances are similarly defined over a $20 \times 20 \text{ km}^2$ grid with blocks of $100 \times 100 \text{ m}^2$. In all cases, customer locations are randomly assigned to unique corner points of these blocks without repetition. Additionally, each instance shares an identical depot position.

Vehicle Specifications: Robot & Truck			
ROBOT		TRUCK	
Medium/Large Instances	Tiny/Small Instances	Medium/Large Instances	Tiny/Small Instances
15 km/h	15 km/h	25 km/h	25 km/h
3 kWh	1.125 kWh	60 kWh	5.625 kWh
0.1 kWh/km	0.1 kWh/km	0.2 kWh/km	0.2 kWh/km

Fig. 6. Vehicle parameters for different instance scales.

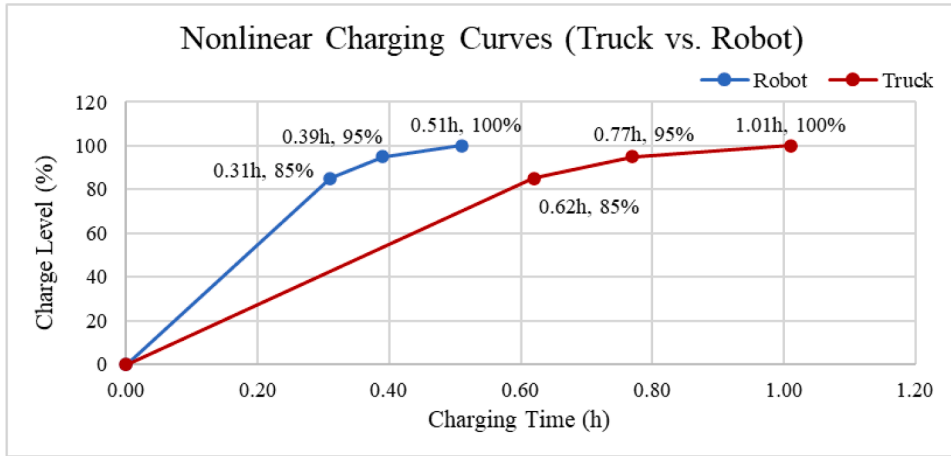


Fig. 7. Nonlinear charging function.

The parameters for the electric vehicles, based on real-world specifications, are detailed in Fig. 6. We observed that for tiny and small-scale instances, which are computationally tractable for the MILP solver, a vehicle with a realistic battery capacity can often complete its entire tour without needing to recharge. While this reflects certain real-world scenarios, it prevents a thorough evaluation of the model's charging-related decision-making logic. To address this and ensure our model's charging components are rigorously tested, we intentionally reduced the battery capacity for these smaller instances. This adjustment was carefully calibrated to make charging a necessary action for finding a feasible solution, thereby enabling a meaningful validation of our algorithm and an effective sensitivity analysis. As a baseline for feasibility, the modified battery capacity was still sufficient to allow a vehicle to complete a round trip between the two most distant nodes in the network.

The basic nonlinear charging function and the number of segments we use in this article are shown in the following Fig. 7.

5.2. Comparative performance of sequential and integrated charging strategies

We consider two charging-decision frameworks: the sequential (STC) approach, which decouples routing and charging decisions, and the integrated (SAC) approach, in which routing and charging are optimized jointly. We first identify the more suitable charging-completion method within the sequential "sequence first, charging second" (STC) framework. Next, after fixing the charging-completion method, we compare two sequencing strategies within STC. Finally, after determining the strongest STC variant, we compare it with the integrated "simultaneous sequencing and charging" (SAC) framework.

First, we compare the charging methods under the STC framework. In the STC framework, charging truck routes are generated using exact or heuristic algorithms. Charging robot routes are determined exclusively via heuristic methods. Charging decisions are made once based on these predetermined routes. To evaluate the effectiveness of these decisions, we added an external testing module that independently applies two charging strategies (the *frvcpy* algorithm and our heuristic) after each destroy-repair cycle in the routing phase. Importantly, this module operates outside the main optimization loop and does not affect route generation.

During testing, we record the following metrics for both frameworks: (i) **Win count**: Among all instances yielding feasible solutions under both approaches, the number of instances in which one strategy attains a superior objective value. (ii) **Run Time**: The average time required by each approach to finalize a charging decision. This testing protocol facilitates a comprehensive comparison of sequential and integrated strategies in terms of charging-decision quality and computational efficiency.

Table 2 shows the experimental results and a comparison of different charging decision methods (*frvcpy* method and heuristic method) on STC framework.

Table 2

Comparison of charging decision methods in STC framework on small scale instance.

Metric	frvcpy method (Mean $\pm$ SD)	Heuristic method (Mean $\pm$ SD)	<i>p</i> -value
Wins in 200,000 times	55,023 $\pm$ 19,379	19,789 $\pm$ 16,862	< 0.001
Run Time (s)	0.0139 $\pm$ 0.0017	0.0003 $\pm$ 0.0000	< 0.001

Note: Results are reported as mean $\pm$ standard deviation. Each of the 20 problem instances was executed 10 times, with 200,000 iterations per run. Data were recorded every 10 iterations. The *p*-values are based on paired t-tests.

Table 3

Performance comparison between Savings-STC and ALNS-STC on small instances with 200,000 iterations.

Instance	Savings-STC		ALNS-STC		Gap (%)
	Time (s)	Obj.	Time (s)	Obj.	
S-01	0.29	3.114	243.16	1.938	60.70
S-02	0.28	2.499	239.91	1.974	26.63
S-03	0.28	3.318	241.31	2.173	52.68
S-04	0.28	2.939	242.64	2.506	17.26
S-05	0.30	3.300	241.52	2.066	59.69
S-06	0.28	3.032	241.83	2.088	45.21
S-07	0.28	2.685	240.14	1.939	38.48
S-08	0.29	2.936	240.43	2.067	42.02
S-09	0.28	3.629	241.55	2.329	55.84
S-10	0.28	3.484	243.00	2.190	59.13
S-11	0.27	3.394	237.95	2.524	34.47
S-12	0.28	3.981	242.69	2.135	86.46
S-13	0.28	2.329	240.65	1.593	46.18
S-14	0.29	3.248	240.33	2.155	50.73
S-15	0.28	2.941	242.01	2.046	43.77
S-16	0.28	2.487	241.65	1.940	28.20
S-17	0.28	3.007	239.39	2.066	45.58
S-18	0.28	3.169	241.34	2.234	41.89
S-19	0.28	2.729	242.11	1.899	43.68
S-20	0.28	3.639	241.22	2.486	46.36
Average	0.28	3.093	241.24	2.117	46.25

Note: Obj. is the average objective value. Gap (%) is calculated as $(Obj_{Savings} - Obj_{ALNS}) / Obj_{ALNS} \times 100\%$.

Each of the 20 problem instances was executed 10 times, with 200,000 iterations per run.

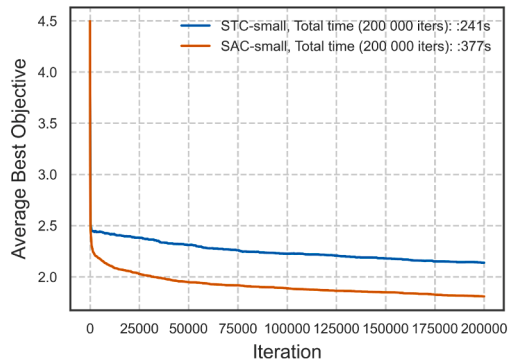
As shown in Table 2, both runtime and win count differ significantly between the frvcpy and heuristic methods ($p < 0.001$). Despite the frvcpy method achieving more wins (55023:19789), its runtime is approximately 46 times longer. Given the high frequency of charging operations in SAC, the heuristic method offers a more scalable and practical solution. We thus adopt the heuristic approach for subsequent experiments.

Second, we compare the sequencing methods under the STC framework. Since both methods share the same charging completion module, the comparison here isolates the impact of the sequencing mechanism.

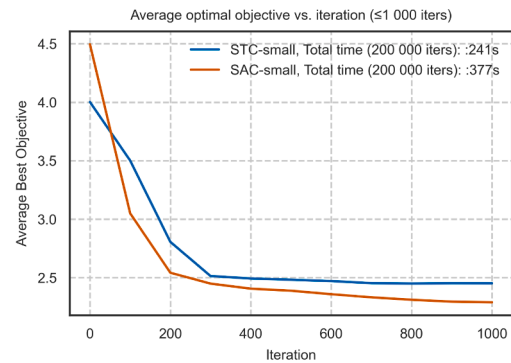
Table 3 summarizes the performance comparison between the Savings-STC and ALNS-STC methods. Table reveals a classic trade-off between computational effort and solution quality. While Savings-STC rapidly generates baseline feasible solutions in just 0.28 s on average, ALNS-STC significantly optimizes the objective (2.117 vs. 3.093). This yields an average relative improvement of 46.25% at the cost of increased, yet acceptable, computational time (~ 241 s for 200,000 iterations). We thus adopt the ALNS-STC (Hereafter referred to collectively as STC) approach for subsequent experiments.

Third, we compare STC and SAC methods. For our experiment setup, we used small-scale instances. We set the stop criteria at 200,000 iterations. Each instance was run 10 times, and then we took the average objective value and the runtime.

Fig. 8 compares the average objective values of STC and SAC over iterations. The results are averaged over ten runs for each of the twenty instances. In Fig. 8a, SAC outperforms STC consistently across the full iteration range (up to 200,000) by converging to lower objective values more quickly and maintaining its advantage throughout. Fig. 8b highlights the first 1000 iterations, during which SAC rapidly reduces the objective value. This demonstrates that SAC achieves faster and more significant improvements in the early stages compared to STC. These advantages make SAC a more effective strategy, justifying its use in subsequent experimental analyses.



(a) Comparison of average objective value over iteration



(b) Comparison of average objective value over iteration (< 1000)

Fig. 8. Comparison of average objective values: (a) overall iterations, (b) iterations < 1000.**Table 4**

Comparison of computational results with and without time valid inequalities.

Metric	Tiny-Scale Instances		Small-Scale Instances	
	With TVIs	Without TVIs	With TVIs	Without TVIs
Avg. Upper Bound	1.087	1.092	1.650	3.023
Avg. Lower Bound	1.087	0.391	1.173	0.056
Avg. Gap	0%	63.60%	27.48%	97.94%
Avg. CPU Time (s)	2.12	6915	7200	7200
# Not Solved to Optimality	0	18	20	20
# No Solution Found	0	0	0	9

Note: Here we limit each charging station to be accessed only once by each vehicle (truck/robot).

5.3. Comparative analysis of MILP and ALNS approaches

5.3.1. Impact of time-dependent valid inequalities

To evaluate the computational impact of the proposed time-dependent valid inequalities (TVIs), we conduct an ablation study. We compare the performance of our base MILP formulation against an enhanced version that incorporates the TVIs, specifically constraints (36)–(37). Both model variants were tested on benchmark instance sets T-01–T-20 and S-01–S-20, with the restriction that each vehicle may visit any charging station at most once. The experiments were run with a time limit of 7200 s. Table 4 summarizes the aggregated computational results, highlighting the effectiveness of the TVIs across standard performance metrics.

The results demonstrate that the inclusion of TVIs yields substantial performance gains. For the tiny-scale instances, the impact is transformative. The base model (without TVIs) fails to solve 18 of the 20 instances to optimality within the time limit, terminating with an average optimality gap of 63.60%. In stark contrast, the TVIs-enhanced model solves all 20 instances to proven optimality in an average of just 2.12 s. This dramatic improvement stems from the TVIs' ability to significantly strengthen the linear programming relaxation, as evidenced by the tight average lower bound (1.087) compared to the base model's weak bound (0.391).

The benefits of the TVIs are also pronounced on the more challenging small-scale instances. The base model (without TVIs) struggles severely, failing to solve any instance to optimality and, more critically, failing to find any feasible solution for 9 of the 20 instances. It terminates with a near-total average optimality gap of 97.94%, reflecting a very weak lower bound of 0.056. While the TVIs-enhanced model also fails to prove optimality for these instances within the time limit, it achieves a vast improvement in solution quality. It successfully finds feasible solutions for all instances and reduces the average optimality gap from 97.94% to 27.48%. This is primarily due to the much stronger dual bound (Avg. LB of 1.173) generated by the TVIs.

5.3.2. Comparing MILP (CPLEX) and ALNS

To assess the scalability of the exact approach, in addition to the tiny- and small-scale experiments we also attempted to solve the medium-scale instances with the proposed MILP model. However, CPLEX failed to obtain any feasible solution within the 7,200-s time limit for all 20 medium-scale instances. These results indicate that, for medium- and large-scale problems, the MILP model is unable to provide valid solutions within a reasonable timeframe. Therefore, the use of heuristic methods, such as ALNS, is not merely a trade-off but a necessity for solving these problems efficiently. In the remainder of this subsection, we focus on a detailed comparison between MILP (CPLEX) and ALNS on tiny- and small-scale instances, where the MILP model remains computationally tractable.

Table 5
Comparison of CPLEX and ALNS performance on T and S instances.

Tiny-Scale Instances					Small-Scale Instances				
Instance	CPLEX		ALNS		Instance	CPLEX		ALNS	
	UB	Time(s)	Min Obj.	Avg Time(s)		UB	Time(s)	Min Obj.	Avg Time(s)
T-01	0.952	0.80	0.952	66.63	S-01	1.436	7200.00	1.407	377.21
T-02	1.072	1.17	1.072	79.52	S-02	1.719	7200.00	1.469	352.98
T-03	0.784	0.31	0.784	54.11	S-03	2.159	7200.00	1.789	417.39
T-04	1.072	3.14	1.072	112.25	S-04	1.592	7200.00	1.576	462.94
T-05	1.249	3.05	1.249	125.43	S-05	1.803	7200.00	1.735	427.99
T-06	1.054	0.81	1.054	81.14	S-06	1.649	7200.00	1.552	344.36
T-07	1.088	2.81	1.088	85.18	S-07	1.394	7200.00	1.446	306.09
T-08	1.209	1.42	1.209	85.05	S-08	1.657	7200.00	1.552	401.16
T-09	1.004	2.64	1.004	63.90	S-09	1.812	7200.00	1.710	396.02
T-10	1.157	4.34	1.157	120.58	S-10	1.987	7200.00	1.962	457.90
T-11	1.305	3.25	1.311	116.08	S-11	2.264	7200.00	2.040	435.28
T-12	1.364	4.97	1.364	92.10	S-12	1.504	7200.00	1.499	381.75
T-13	0.984	0.33	0.984	94.56	S-13	1.187	7200.00	1.187	296.15
T-14	0.976	0.86	0.976	71.19	S-14	1.565	7200.00	1.657	367.93
T-15	1.040	0.95	1.040	86.31	S-15	1.407	7200.00	1.552	376.73
T-16	1.262	5.59	1.262	74.44	S-16	1.473	7200.00	1.548	362.55
T-17	0.984	1.13	0.984	88.00	S-17	1.606	7200.00	1.495	332.69
T-18	1.224	3.98	1.224	111.87	S-18	1.539	7200.00	1.568	383.10
T-19	1.040	0.38	1.040	138.99	S-19	1.354	7200.00	1.433	357.25
T-20	0.911	0.55	0.911	82.38	S-20	1.895	7200.00	1.887	413.52
Avg	1.087	2.12	1.087	91.49	Avg	1.650	7200.00	1.603	382.55

Note: Here we limit each charging station to be accessed only once by each vehicle (truck/robot) for both CPLEX and ALNS model. CPLEX uses 32 cores, while ALNS uses a single core.

The objective of this comparison is to evaluate the practical feasibility and performance of MILP (CPLEX) and ALNS in solving ETRD-NL problem on tiny/small-scale problem, particularly in terms of computational efficiency, and solution quality across different problem sizes.

We limit each charging station to be visited only once by the same vehicle. Allowing multiple visits to the same station would significantly increase the number of variables in the MILP model, making it hard to solve efficiently even for tiny-scale instances. To ensure comparability, we imposed the same constraint on both approaches by disallowing repeated visits to the same charging station. In the MILP model, this is encoded as a hard constraint, while in the ALNS framework, a large penalty is added to the objective function for repeated visits.

Table 5 shows the results of comparison of CPLEX and ALNS performance.

Table 5 indicates that for tiny instances, CPLEX yield optimal solutions and CPLEX with high computational efficiency (average runtime of 2.12 s). ALNS failed to find the optimal solution for instance T-11. We found that it identified the optimal route, but a greedy charging rule prevented it from reaching optimality. For small-scale instances, all CPLEX cases couldn't be solved within 2 h, so we report the best upper bounds it found. On average, CPLEX achieved a solution quality of 1.650, slightly worse than ALNS's 1.603 with 382.55s runtime, resulting in an average gap of about 2.8%. If the problem size increases further, it's clear that CPLEX struggles to keep up, making ALNS a more suitable choice. Note a key limitation of our algorithm is that the greedy charging strategy does not ensure an optimal charging schedule, even for an optimal route. This may prevent our method from guaranteeing the minimization of the time objective.

5.3.3. Evaluation of ALNS for medium/large scale instances

In order to assess the behavior of our ALNS heuristic on medium/larger problem instances, Fig. 9 plots the evolution of the average best-found objective (20 instances and each run 10 times) as a function of iteration count, separately for medium-scale and larger-scale instances.

Fig. 9 illustrates the continuous convergence of the objective function achieved by our proposed ALNS. As shown in Fig. 9a and b, the average computation time for 200,000 iterations is 1252 s for medium-scale instances and 19,104 s for large-scale instances, respectively. Notably, the convergence curve reveals a point of diminishing returns: the improvement gained during the final 100,000 iterations is marginal, accounting for less than 5% of the total improvement. This observation suggests that an appropriate number of iterations can be selected to strike a practical balance between solution quality and computational time.

5.4. Sensitivity analysis on charging mode and system parameters

We conduct a series of sensitivity analysis experiments on tiny-scale instances to evaluate the influence of charging modes and several key system parameters: battery capacity, charging rate, and the number of charging stations.

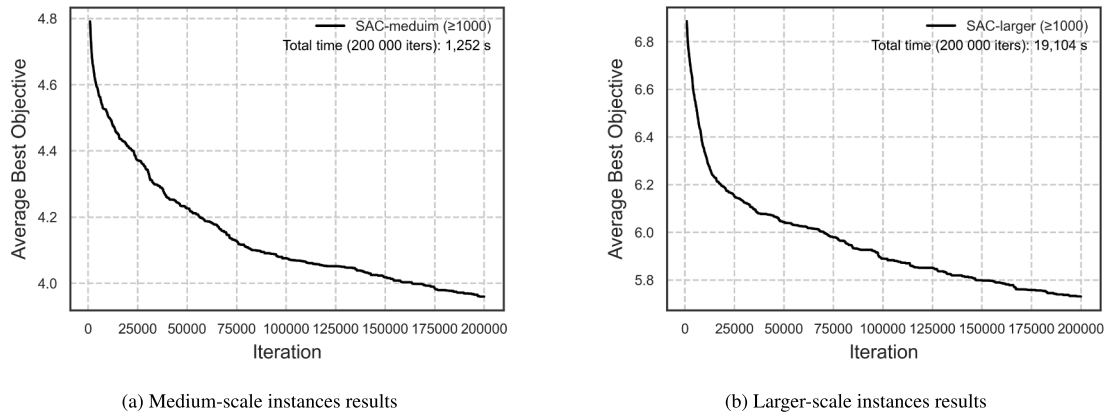


Fig. 9. Average objective values over iteration on medium/larger-scale instances.

Table 6

Sensitivity analysis results for charging mode, battery capacity, charging rate, and number of charging stations.

Experiment Type	Instance	LFC		NFC		LPC		NPC	
		Value	$\Delta(\%)$	Value	$\Delta(\%)$	Value	$\Delta(\%)$	Value	$\Delta(\%)$
Charging modes	T-5.625-1.125-1-1 (baseline)	1.162	0.0	1.228	0.0	1.092	0.0	1.086	0.0
Battery Capacity	T-3.5-0.75-1-1	1.958	68.5	2.063	68.0	1.820	66.7	1.671	53.9
	T-7.5-1.5-1-1	1.056	-9.1	1.078	-12.2	1.036	-5.1	1.035	-4.7
Charging Rate	T-5.625-1.125-0.5-1	1.216	4.7	1.350	9.9	1.125	3.0	1.113	2.5
	T-5.625-1.125-2-1	1.119	-3.7	1.154	-6.0	1.078	-1.3	1.075	-1.0
Station Number	T-5.625-1.125-1-1/3	1.441	24.0	1.553	26.5	1.371	25.6	1.342	23.6
	T-5.625-1.125-1-2/3	1.282	10.3	1.388	13.0	1.228	12.5	1.207	11.1

Note: $\Delta(\%) = \frac{\text{value} - \text{baseline}}{\text{baseline}} \times 100\%$.

LFC, NFC, LPC, and NPC represent the objective values under different charging function and modes.

Each charging station is restricted to be accessed only once by each vehicle (truck/robot).

T-X-Y-Z-W denotes instance with truck battery capacity X, robot battery capacity Y, charging rate Z, and relative number of stations W (if applicable).

The baseline configuration is T-5.625-1.125-1-1.

We will evaluate the impact of four charging modes on the objective function: nonlinear full charging mode (NFC), nonlinear partial charging mode (NPC), linear full charging mode (LFC), and linear partial charging mode (LPC). Comparisons among these four settings serve as ablation studies to isolate and verify the specific impacts of charging linearity (linear vs. nonlinear) and charging flexibility (full vs. partial) on system performance. Furthermore, we analyze the effects of varying battery capacity and charging rate by simultaneously scaling the values for both trucks and robots to 50% and 200% of their respective baselines. Finally, we investigate the influence of facility availability by reducing the number of charging stations to two-thirds and one-third of the original count, ensuring that the selected stations are a subset of the initial set.

To ensure that observed changes in the objective function are attributable solely to parameter variations, the sensitivity analysis must be based on high-quality solutions, ideally optimal ones. This requirement is critical in our study, as switching between different charging modes (e.g., partial charging vs. full charging) need to make significant changes to the algorithm, especially heuristic policy to charge. Relying on different tailored heuristics for each mode could introduce performance biases, making it difficult to isolate the true effect of the charging mode itself. Therefore, we used CPLEX to solve tiny-scale instances, ensuring a fair and unbiased comparison.

Table 6 summarizes the objective values and their percentage changes relative to the baseline under various parameter settings.

The analysis yields several key insights. First, regarding the charging modes, the results from the baseline instance indicate two main points. Firstly, partial charging modes outperform full charging modes, as evidenced by the lower objective values for LPC (1.092) compared to LFC (1.162), and for NPC (1.086) compared to NFC (1.228). Secondly, the results highlight a crucial interaction between the charging function (linear vs. nonlinear) and the charging mode (full vs. partial). While the linear model (LFC) appears more efficient under the rigid full-charging constraint (1.162 vs. 1.228), the nonlinear model (NPC) demonstrates its superiority in the more practical partial-charging scenario (1.086 vs. 1.092). Real-world battery charging is nonlinear, often characterized by a high charging rate at a low state of charge, which then tapers off. Therefore, solutions derived from linear models, may prove to be infeasible when implemented in practice. We provide a detailed example of an optimal solution (T-10) under different charging

modes in [Appendix C](#), illustrating the routes as well as the charging amounts and time allocations, to help readers more intuitively understand the structure of the model and its solutions.

The experiments on battery capacity were conducted by scaling both the truck and robot batteries to 50% and 200% of the baseline levels. The results confirm that capacity changes have a marked impact on performance: cutting the capacity in half degrades the objective value by more than 50%, whereas doubling it yields only a modest improvement of roughly 5-12%. The intuition is that smaller batteries force vehicles to interrupt their routes more frequently for recharging, which, combined with the capacity constraints, prevents them from choosing shorter or more energy-efficient paths. Analogously, the charging-rate experiments halve the baseline rate to 50% and double it to 200%. Changing the rate in either direction produces far smaller effects than changing capacity: even a two-fold decrease or increase shifts the objective value by less than 10%. In other words, within the tested range, the system is considerably less sensitive to how fast energy can be transferred than to how much energy can be stored. In addition, reducing the number of available charging stations has a pronounced effect, leading to objective value increases of over 10% when reduced to two-thirds and over 20% when reduced to one-third of the baseline. An interesting phenomenon observed across all experiments is the relative sensitivity of the different charging models. The NPC mode consistently demonstrates the lowest sensitivity to variations in parameters, while the NFC model exhibits the highest sensitivity, showing the largest fluctuations in its objective value in response to the same changes.

CRediT authorship contribution statement

Shaohua Yu: Writing – review & editing, Writing – original draft, Software, Methodology, Funding acquisition, Formal analysis, Conceptualization; **Tengkuo Zhu:** Investigation, Formal analysis, Conceptualization; **Jakob Puchinger:** Writing – review & editing, Writing – original draft, Methodology.

6. Conclusion and future work

This paper studies an electric truck-based robot collaborative delivery problem with nonlinear charging functions. We first formulate an exact mixed-integer linear programming model that captures the interdependent routing, scheduling, and charging decisions for this problem. To enhance tractability, a time-based valid inequality was derived. Computational experiments on tiny-scale instances demonstrated that these inequalities reduced CPLEX's solving time by several orders of magnitude (from 6915 s to 2.12 s). As a result, exact optimization became viable for tiny-scale problems.

Two complementary solution paradigms are proposed in addressing larger instances: The sequence-first, charging-second heuristic is a two-phase decomposition-based algorithm, it first determines vehicle sequences and then schedules charging. The simultaneous sequencing and charging: an integrated ALNS that optimizes routes and charging plans in tandem. On tiny-scale benchmarks, the ALNS always attains optimality; on small-scale instances, it yields better upper bounds than CPLEX within a 7,200 s limit and requires, on average, only 382.6 s. These results underscore the ALNS's efficiency and potential for medium/large-scale, real-world deployments.

A thorough sensitivity analysis has yielded two key managerial insights. While a linear charging approximation may appear optimal, it fails to consider the inherently nonlinear nature of battery physics and can result in impractical field implementations. Secondly, system performance is most sensitive to variations in battery capacity and the number of charging stations, whereas variations in charging rate have a comparatively minor impact. Consequently, fleet operators should prioritize investments in higher-capacity batteries and strategically locate charging infrastructure to achieve the most significant improvements in delivery efficiency.

Overall, the modelling advancements, valid inequalities, and heuristic framework presented herein offer a methodology for emerging electric truck-robot delivery operations and lay the groundwork for future extensions to multi-vehicle, multi-robot, dynamic, and stochastic environments. A promising direction for future research involves relaxing the synchronous constraint at charging stations: trucks and robots can autonomously choose whether to meet at a charging station or rendezvous with each other, provided they each visit a charging station independently. This extension would enhance model realism by accommodating asynchronous arrivals and independent charging behaviors, while addressing the associated computational challenges.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

The research is supported by [National Natural Science Foundation of China](#) [grant no. 72301137].

Appendix A. Adaptive large neighborhood search framework

The ALNS algorithm framework is an efficient and highly regarded method in the field of vehicle routing problems, supported by a large body of literature (Sacramento et al., 2019; Mara et al., 2022).

The proposed ALNS method initializes with a feasible solution, which it iteratively refines until satisfying a predetermined termination condition. In this study, the termination condition is defined by the maximum iteration count (denoted as *MaxIteNum*). Within each iteration, the current solution undergoes systematic perturbation through a destroy operator, which strategically removes selected nodes or routes, thereby creating partial solutions. Subsequently, a repair operator reconstructs the partial solution by reinserting the removed elements according to heuristic principles, aiming to yield a solution of enhanced quality. To prevent stagnation, if no solution improvement occurs within a pre-specified number of iterations (*MaxNonImp*), the search restarts from a newly generated initial solution.

Ensuring feasibility throughout reconstruction necessitates a systematic route evaluation approach embedded within the repair operations. Additionally, the search incorporates a simulated annealing-inspired acceptance criterion, designed to facilitate exploration of potentially inferior solutions, thereby enhancing the diversity and robustness of the search. The adaptive nature of the framework is further reinforced through a dynamic weight-adjustment mechanism, where the performance of each destroy-and-repair pair informs continual recalibration of operator weights. This adaptive weighting scheme enables the search process to dynamically respond to the evolving characteristics of the problem instance and the ongoing search trajectory, aligning with the methods outlined in Pisinger and Ropke (2018).

Within the simulated annealing framework, the acceptance probability is modulated by a temperature parameter, denoted as T . Specifically, a candidate solution x^t is deterministically accepted if its cost is lower than or equal to the incumbent best solution x^b , i.e., if $c(x^t) \leq c(x^b)$. Conversely, inferior solutions are probabilistically accepted based on the acceptance function $\exp\left(\frac{c(x^b) - c(x^t)}{T}\right)$, wherein $T > 0$ represents the current temperature level. The search commences with an initial temperature setting $T_{st} > 0$, progressively reduced at each iteration via the cooling schedule $T \leftarrow \alpha \times T$, with cooling factor $\alpha \in [0, 1]$.

Algorithm 4 formalizes the proposed ALNS procedure. Initially, the optimal solution found is maintained in the variable *recordset* (Line 1). The search initiates from an initial solution x , simultaneously recorded as the current best solution x^b (Line 2). An iteration counter for non-improving steps (*NumNonImp*) is initialized to zero (Line 3) and increments whenever subsequent iterations fail to improve the solution quality. This iterative process, comprising consecutive destroy-and-repair operations, continues until meeting the specified maximum iteration limit (*MaxIteNum*) (Lines 4-19). Upon exceeding the non-improvement threshold (*MaxNonImp*), the algorithm triggers a *restart()* procedure, reinitializing the search from a newly generated feasible solution, though the iteration counter remains unaffected.

Algorithm 4 ALNS.

Require: Problem instance I ; Set of destroy operators D ; Set of repair operators R

Ensure: Best-found solution x^b

```

1:  $x \leftarrow \text{INITIALSOLUTION}(I)$                                 ▷ Generate initial solution
2:  $x^b \leftarrow x$                                               ▷ Initialize global best solution
3:  $S_{\text{record}} \leftarrow \emptyset$                                   ▷ Initialize record set for restarts
4:  $n_{\text{non\_improving}} \leftarrow 0$                                ▷ Counter for non-improving iterations
5: while stopping criterion not met do
6:   Select a destroy operator  $d \in D$  and a repair operator  $r \in R$ 
7:    $x' \leftarrow r(d(x))$                                        ▷ Generate a new candidate solution
8:   if  $\text{ACCEPT}(x', x, T)$  then                                  ▷ e.g., Simulated Annealing criterion
9:      $x \leftarrow x'$ 
10:  end if
11:  if  $f(x) < f(x^b)$  then                                       ▷ Found a new best solution?
12:     $x^b \leftarrow x$ 
13:     $n_{\text{non\_improving}} \leftarrow 0$ 
14:  else
15:     $n_{\text{non\_improving}} \leftarrow n_{\text{non\_improving}} + 1$ 
16:    if restart condition is met then
17:      Add  $x^b$  to  $S_{\text{record}}$ 
18:       $x \leftarrow \text{RESTART}$ 
19:    end if
20:  end if
21:   $\text{UPDATEWEIGHTS}(D, R)$                                        ▷ Update operator selection weights
22:   $\text{UPDATEPARAMETERS}$                                            ▷ e.g., update temperature  $T$ 
23: end while
24: return  $x^b$ 

```

Appendix B. Proof of proposition: charge-enough principle

Proof. Let two consecutive charging stations on the route be i and j , followed by station k . Let the energy required to travel the legs $i \rightarrow j$ and $j \rightarrow k$ be C_{ij} and C_{jk} , respectively. Assume the vehicle arrives at station i with residual energy $e_i < C_{ij}$.

We compare two charging strategies for the segment covering stations i and j . Crucially, we enforce that under both strategies, the vehicle departs station j with exactly C_{jk} energy. This ensures the vehicle's state is identical for the remainder of the journey, allowing for a fair comparison of the time spent charging at i and j .

Strategy A (Over-charging). At station i , charge more than is immediately necessary. Specifically, charge an amount to reach an SoC of $C_{ij} + \Delta$ for some $\Delta > 0$. The vehicle arrives at station j with residual energy Δ . At station j , it then charges from Δ up to the required level C_{jk} .

Strategy B (enough-charging). At station i , charge only the amount needed to reach j . The vehicle departs with SoC C_{ij} and arrives at j with zero energy. At station j , it then charges from 0 up to C_{jk} .

Because $T(s)$ is the time to charge from 0 to s , the total charging times for these strategies across stations i and j are:

$$t_A = [T(C_{ij} + \Delta) - T(e_i)] + [T(C_{jk}) - T(\Delta)],$$

$$t_B = [T(C_{ij}) - T(e_i)] + [T(C_{jk}) - T(0)].$$

Note that in both cases, the vehicle departs j with SoC C_{jk} .

We compute the difference in total time:

$$\begin{aligned} t_A - t_B &= ([T(C_{ij} + \Delta) - T(e_i)] + [T(C_{jk}) - T(\Delta)]) \\ &\quad - ([T(C_{ij}) - T(e_i)] + [T(C_{jk}) - T(0)]) \\ &= T(C_{ij} + \Delta) - T(\Delta) - T(C_{ij}) + T(0). \end{aligned}$$

Since $T(0) = 0$, this simplifies to:

$$t_A - t_B = [T(C_{ij} + \Delta) - T(C_{ij})] - [T(\Delta) - T(0)].$$

This expression compares the time taken to add Δ units of energy starting from a high SoC (C_{ij}) versus starting from zero.

Define a function $h(s) = T(s + \Delta) - T(s)$. The convexity of T implies that its derivative $T'(s)$ is non-decreasing. Therefore, the derivative of $h(s)$ is:

$$h'(s) = T'(s + \Delta) - T'(s) \geq 0.$$

This shows that $h(s)$ is a non-decreasing function of s . Since $C_{ij} \geq 0$, it follows that $h(C_{ij}) \geq h(0)$. Substituting the definition of $h(s)$ gives:

$$T(C_{ij} + \Delta) - T(C_{ij}) \geq T(0 + \Delta) - T(0).$$

Therefore, $t_A - t_B \geq 0$. If T is strictly convex ($T'' > 0$) and $\Delta > 0$, then T' is strictly increasing, making $h'(s) > 0$, so the inequality is strict: $t_A > t_B$. $\square$

Appendix C. Example of route and charging decisions under four policies: LFC, LPC, NFC, and NPC with Instance T-10.

This composite example, Fig. C.1, is provided to visually illustrate, for instance T-10, the representative optimal route structures of the truck-robot collaborative delivery system under four charging scenarios (linear/nonlinear $\times$ full/partial charging) and their corresponding objective values. It also reports, at each charging station, the charging decisions in terms of charged energy and charging time. We first present the representative route under the NPC mode in Fig. C.1d. The route patterns and interpretations for the other modes (LFC in Fig. C.1a, LPC in Fig. C.1b, and NFC in Fig. C.1c) follow the same logic and are therefore omitted for brevity.

Fig. C.1d illustrates the solution for the electric truck-based robot delivery system. The truck departs from the depot node 0 carrying the robot and travels to the first customer, node 4. After arriving at node 4, the truck continues its tour, while the robot is deployed from the truck and starts its independent route.

After serving customer 4, the truck follows the route $4 \rightarrow 6 \rightarrow 5 \rightarrow 2 \rightarrow 1 \rightarrow 3 \rightarrow 9$. It then waits at node 9 to rendezvous with the robot, and finally returns to the depot (denoted as 0') with the robot onboard. The energy-feasibility check shows that the truck's state of charge remains positive throughout the route; therefore, the truck does not need to recharge at any charging station.

After separating from the truck at node 4, the robot first travels to charging station 10. At station 10, it recharges by 0.175 units of energy, which takes 0.057 units of charging time. This partial recharge ensures that the robot can complete the subsequent legs and reach station 9 with zero remaining energy. The robot then travels along $10 \rightarrow 7 \rightarrow 9$. The truck and the robot meet at charging station 9, after which the truck carries the robot back to the depot.

We then further examine, in Fig. C.1e, the charged energy and charging time of the truck and the robot at charging stations under different charging modes.

Fig. C.1e shows that the robot's charging decisions reveal a representative phenomenon. Under the partial-charging scenarios (LPC vs. NPC), when the robot replenishes the same amount of energy (0.175) at the charging station, the charging time in LPC is actually

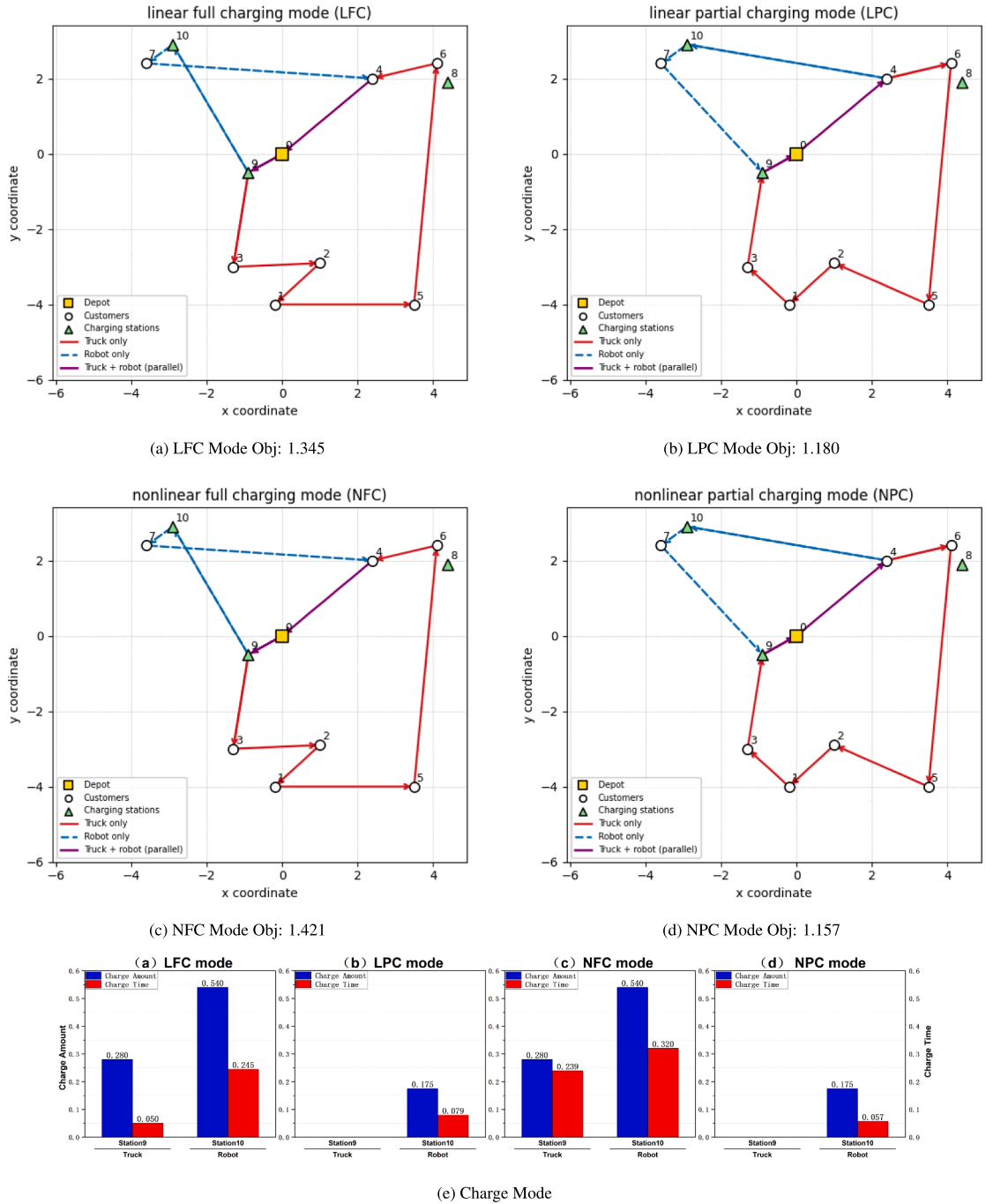


Fig. C.1. An example of route and charging decisions under four policies: LFC, LPC, NFC, and NPC with Instance T-10.

longer than that in NPC; whereas under the full-charging scenarios (LFC vs. NFC), when the robot replenishes the same amount of energy (0.540), the charging time in LFC is shorter than that in NFC. This behavior stems from the nature of the nonlinear charging function: it exhibits different effective charging slopes across different SOC ranges, which causes the “charging time for the same charged energy” to deviate from the linear-model estimate in a non-uniform manner—nonlinear charging can be slower in some slope segments, yet potentially faster in others. Therefore, if a linear charging function is used to approximate the real nonlinear charging process, then in those SOC ranges where the linear model underestimates charging time, the algorithm may incorrectly assume during planning that sufficient energy can be obtained within the allotted time. Once implemented with the true charging dynamics, the

actually acquired energy may be insufficient to support the subsequent travel, thereby violating the energy constraints and rendering the planned route infeasible.

References

- Aghalari, A., Salamah, D.E., Marino, C., Marufuzzaman, M., 2023. Electric vehicles fast charger location-routing problem under ambient temperature. *Ann. Oper. Res.* 324 (1), 721–759.
- Alarcón, F.E., Mac Cawley, A., Sauma, E., 2023. Electric mobility toward sustainable cities and road-freight logistics: a systematic review and future research directions. *J. Clean. Prod.* 430, 138959.
- Alverhed, E., Hellgren, S., Isaksson, H., Olsson, L., Palmqvist, H., Flodén, J., 2024. Autonomous last-mile delivery robots: a literature review. *Eur. Transp. Res. Rev.* 16 (1), 4. <https://doi.org/10.1186/s12544-023-00629-7>
- Bakach, I., Campbell, A.M., Ehmk, J.F., 2021. A two-tier urban delivery network with robot-based deliveries. *Networks* 78 (4), 461–483.
- Boysen, N., Schwerdfeger, S., Weidinger, F., 2018. Scheduling last-mile deliveries with truck-based autonomous robots. *Eur. J. Oper. Res.* 271 (3), 1085–1099.
- Carlsson, J.G., Song, S., 2018. Coordinated logistics with a truck and a drone. *Manag. Sci.* 64 (9), 4052–4069.
- Cavani, S., Iori, M., Roberti, R., 2021. Exact methods for the traveling salesman problem with multiple drones. *Transp. Res. C Emerg. Technol.* 130, 103280.
- Chen, C., Demir, E., Huang, Y., 2021. An adaptive large neighborhood search heuristic for the vehicle routing problem with time windows and delivery robots. *Eur. J. Oper. Res.* 294 (3), 1164–1180.
- Chung, S.-H., 2021. Applications of smart technologies in logistics and transport: a review. *Transp. Res. E Logist. Transp. Rev.* 153, 102455.
- Desaulniers, G., Errico, F., Irnich, S., Schneider, M., 2016. Exact algorithms for electric vehicle-routing problems with time windows. *Oper. Res.* 64 (6), 1388–1405.
- Doppstadt, C., Koberstein, A., Vigo, D., 2020. The hybrid electric vehicle-traveling salesman problem with time windows. *Eur. J. Oper. Res.* 284 (2), 675–692.
- Froger, A., Jabali, O., Mendoza, J.E., Laporte, G., 2022. The electric vehicle routing problem with capacitated charging stations. *Transp. Sci.* 56 (2), 460–482.
- Froger, A., Mendoza, J.E., Jabali, O., Laporte, G., 2019. Improved formulations and algorithmic components for the electric vehicle routing problem with nonlinear charging functions. *Comput. Oper. Res.* 104, 256–294.
- Goeke, D., 2019. Granular tabu search for the pickup and delivery problem with time windows and electric vehicles. *Eur. J. Oper. Res.* 278 (3), 821–836.
- Ham, A.M., 2018. Integrated scheduling of m-truck, m-drone, and m-depot constrained by time-window, drop-pickup, and m-visit using constraint programming. *Transp. Res. C Emerg. Technol.* 91, 1–14.
- Han, B., Chi, T., Sun, Z., 2025. Collaborative delivery problem of trucks and multiple drones: an en-route operation with flexible launch positions. *Int. Trans. Oper. Res.* . <https://doi.org/10.1111/itor.70045>
- Heimfarth, A., Ostermeier, M., Hübner, A., 2022. A mixed truck and robot delivery approach for the daily supply of customers. *Eur. J. Oper. Res.* 303 (1), 401–421.
- Hiermann, G., Puchinger, J., Ropke, S., Hartl, R.F., 2016. The electric fleet size and mix vehicle routing problem with time windows and recharging stations. *Eur. J. Oper. Res.* 252 (3), 995–1018.
- Hulagu, S., Celikoglu, H.B., 2021. Electric vehicle location routing problem with vehicle motion dynamics-based energy consumption and recovery. *IEEE Trans. Intell. Transp. Syst.* 23 (8), 10275–10286.
- Jie, W., Yang, J., Zhang, M., Huang, Y., 2019. The two-echelon capacitated electric vehicle routing problem with battery swapping stations: formulation and efficient methodology. *Eur. J. Oper. Res.* 272 (3), 879–904.
- Kang, M., Lee, C., 2021. An exact algorithm for heterogeneous drone-truck routing problem. *Transp. Sci.* 55 (5), 1088–1112.
- Kullman, N.D., Froger, A., Mendoza, J.E., Goodson, J.C., 2021a. frcvpy: an open-source solver for the fixed route vehicle charging problem. *INFORMS J. Comput.* 33 (4), 1277–1283.
- Kullman, N.D., Goodson, J.C., Mendoza, J.E., 2021b. Electric vehicle routing with public charging stations. *Transp. Sci.* 55 (3), 637–659.
- Liu, C., Xu, Z., Xu, Z., Zhang, M., Gao, Y., Wang, J., Qu, X., 2025. Joint route and charging schedule optimization for electric vehicles considering nonlinear charging process using state-space-time networks. *Transp. Res. E Logist. Transp. Rev.* 201, 104233.
- Luo, H., Duan, J., Wang, G., 2025. Mathematical models for truck-drone routing problem: literature review. *Appl. Math. Model.* , 116074.
- Madani, B., Ndiaye, M., Salhi, S., 2024. Hybrid truck-drone delivery system with multi-visits and multi-launch and retrieval locations: mathematical model and adaptive variable neighborhood search with neighborhood categorization. *Eur. J. Oper. Res.* 316 (1), 100–125.
- Mara, S. T.W., Norcahyo, R., Jodiawan, P., Lusiantoro, L., Rifai, A.P., 2022. A survey of adaptive large neighborhood search algorithms and applications. *Comput. Oper. Res.* 146, 105903.
- Moghadam, A.M., Salhi, A., Yang, X., Nguyen, T.T., Pourhejazy, P., 2025. A multi-objective approach for the integrated planning of drone and robot assisted truck operations in last-mile delivery. *Expert Syst. Appl.* 269, 126434. <https://doi.org/10.1016/j.eswa.2025.126434>
- Montoya, A., Guéret, C., Mendoza, J.E., Villegas, J.G., 2017. The electric vehicle routing problem with nonlinear charging function. *Transp. Res. B Methodol.* 103, 87–110.
- Murray, C.C., Chu, A.G., 2015. The flying sidekick traveling salesman problem: optimization of drone-assisted parcel delivery. *Transp. Res. C Emerg. Technol.* 54, 86–109.
- Ostermeier, M., Heimfarth, A., Hübner, A., 2022. Cost-optimal truck-and-robot routing for last-mile delivery. *Networks* 79 (3), 364–389.
- Ostermeier, M., Heimfarth, A., Hübner, A., 2023. The multi-vehicle truck-and-robot routing problem for last-mile delivery. *Eur. J. Oper. Res.* 310 (2), 680–697.
- Ostermeier, M., Huf, T., 2026. The truck-and-robot routing problem with pickups and deliveries. *Eur. J. Oper. Res.* 333(1), 174–191.
- Park, Y., Jeong, J., Wang, C., Woo, S., 2025. Multi-modal electric logistics system: electric trucks as floating chargers for electric drones. *Transp. Res. D Transp. Environ.* 146, 104843.
- Pisinger, D., Ropke, S., 2018. Large neighborhood search. In: Gendreau, M., Potvin, J.-Y. (Eds.), *Handbook of Metaheuristics*. Springer, International Series in Operations Research & Management Science, 272, 99–127. https://doi.org/10.1007/978-3-319-91086-4_4.
- Qin, H., Su, X., Ren, T., Luo, Z., 2021. A review on the electric vehicle routing problems: variants and algorithms. *Front. Eng. Manag.* 8, 370–389.
- Sacramento, D., Pisinger, D., Ropke, S., 2019. An adaptive large neighborhood search metaheuristic for the vehicle routing problem with drones. *Transp. Res. C Emerg. Technol.* 102, 289–315.
- Saputra, H.M., Nor, N. S.M., Rijanto, E., Md Zain, M.Z., Mat Darus, I.Z., Yazid, E., 2024. A review of robotic charging for electric vehicles. *Int. J. Intell. Rob. Appl.* 8 (1), 193–229.
- Schneider, M., Stenger, A., Goeke, D., 2014. The electric vehicle-routing problem with time windows and recharging stations. *Transp. Sci.* 48 (4), 500–520.
- Seyfi, M., Alinaghian, M., Ghorbani, E., Çatay, B., Sabbagh, M.S., 2022. Multi-mode hybrid electric vehicle routing problem. *Transp. Res. E Logist. Transp. Rev.* 166, 102882.
- Srinivas, S., Ramachandiran, S., Rajendran, S., 2022. Autonomous robot-driven deliveries: a review of recent developments and future directions. *Transp. Res. Part E Logist. Transp. Rev.* 165, 102834.
- Tamke, F., Buscher, U., 2021. A branch-and-cut algorithm for the vehicle routing problem with drones. *Transp. Res. B Methodol.* 144, 174–203.
- Tang, X., Li, M., Lin, X., He, F., 2020. Online operations of automated electric taxi fleets: an advisor-student reinforcement learning framework. *Transp. Res. C Emerg. Technol.* 121, 102844.
- Turan, B., Hemmelmayr, V., Larsen, A., Puchinger, J., 2024. Transition towards sustainable mobility: the role of transport optimization. *Cent. Eur. J. Oper. Res.* 32, 435–456. <https://doi.org/10.1007/s10100-023-00888-8>
- Wang, F., Li, H., Xiong, H., 2025. Truck-drone routing problem with stochastic demand. *Eur. J. Oper. Res.* 322(3), 854–869.
- Wang, W., Adulyasak, Y., Cordeau, J.-F., He, G., 2025. The heterogeneous-fleet electric vehicle routing problem with nonlinear charging functions. *Transp. Res. C Emerg. Technol.* 170, 104932.

- Wang, W., Zhao, J., 2023. Partial linear recharging strategy for the electric fleet size and mix vehicle routing problem with time windows and recharging stations. *Eur. J. Oper. Res.* 308 (2), 929–948.
- Xie, D.-F., Yu, Y.-P., Zhou, G.-J., Zhao, X.-M., Chen, Y.-J., 2023. Collaborative optimization of electric bus line scheduling with multiple charging modes. *Transp. Res. D Transp. Environ.* 114, 103551.
- Yamin, D., Desaulniers, G., Mendoza, J.E., 2025. The electric vehicle routing and overnight charging scheduling problem on a multigraph. *INFORMS J. Comput.* 37 (4), 808–830.
- Yang, S., Xia, J., Guo, S., Fang, X., Ni, C., 2025. A novel time-dependent electric vehicle pickup and delivery problem with time windows in cold supply chains. *Expert Syst. Appl.* 294, 128813.
- Yu, S., Campbell, A.M., Ehmke, J.F., Puchinger, J., 2023. Last-mile robot deliveries with access restrictions. In: *ROADEF 2023: 24ème congrès de la Société Française de Recherche Opérationnelle et d'Aide à la Décision*.
- Yu, S., Campbell, A.M., Ehmke, J.F., Puchinger, J., 2026. A hybrid urban delivery system with robots. *Eur. J. Oper. Res.* 331(2), 441–461.
- Yu, S., Puchinger, J., 2024. Collaborative truck-robot deliveries: challenges, models, and methods. *Ann. Oper. Res.* , 1–38 <https://doi.org/10.1007/s10479-023-05757-y>
- Yu, S., Puchinger, J., Sun, S., 2020. Two-echelon urban deliveries using autonomous vehicles. *Transp. Res. E Logist. Transp. Rev.* 141, 102018.
- Yu, S., Puchinger, J., Sun, S., 2022. Van-based robot hybrid pickup and delivery routing problem. *Eur. J. Oper. Res.* 298 (3), 894–914.
- Yu, S., Puchinger, J., Sun, S., 2024. Electric van-based robot deliveries with en-route charging. *Eur. J. Oper. Res.* 317 (3), 806–826.
- Zhou, S., Zhang, D., Yuan, W., Wang, Z., Zhou, L., Bell, M. G.H., 2024. Pickup and delivery problem with electric vehicles and time windows considering queues. *Transp. Res. C Emerg. Technol.* 167, 104829.
- Zhou, Y., Meng, Q., Ong, G.P., 2022. Electric bus charging scheduling for a single public transport route considering nonlinear charging profile and battery degradation effect. *Transp. Res. B Methodol.* 159, 49–75.
- Zhu, T., Boyles, S.D., Unnikrishnan, A., 2025. Plug-in hybrid electric vehicle traveling-salesman problem with drone. *Transp. Res. Rec.* 2679 (3), 438–456.